

1 Reminder 1	10.8 AC Automaton..... 12	2 set nu rnu cul mouse=a cin et ts=2 sw=2 sts=2 autochdir
1.1 Observations and Tricks..... 1	10.9 AC Automaton - Dict Links..... 13	3b=unnamedplus
1.2 Bug List..... 1	10.10 AC Automaton - Counting	3 no ; :
2 Init (Linux) 1	Patterns..... 13	4 ino {<CR> {<CR>}<Esc>ko
2.1 vimrc..... 1	10.11 AC Automaton - Pattern	5 ca Hash w !cpp -dD -P \ tr -d '[:space:]' \ openssl sha256
2.2 bashrc..... 1	Position..... 13	-r \ cut -c-6
3 Basic 1	11 Geometry 13	1 " Gino's .vimrc "
3.1 Template (Using Codebook)..... 1	11.1 Template..... 13	2 "no <C-h> ^ <C-l> \$ h b l e b l e h
3.2 PBDS, Random..... 1	11.2 Basic..... 13	2.2 bashrc
3.3 Debug..... 1	11.3 Line / Segment Intersections..... 13	c08ff4
3.4 SVG Writer..... 1	11.4 Point-Line / Segment Distance..... 13	1 gogo() {
3.5 Python..... 2	11.5 Sort..... 13	2 g++ -std=c++20 -O2 "\$1" && echo run && ./a.out
3.6 Stress Tests..... 2	11.6 Convex Hull..... 14	3}
4 Data Structure 2	11.7 Triangle Centers..... 14	4 alias vig='vim -u ~/.vimrc-gino'
4.1 Mo's Algorithm..... 2	11.8 Point Inside..... 14	3 Basic
4.2 CDQ..... 2	11.9 Point In Triangle..... 14	3.1 Template (Using Codebook)
4.3 Persistent Treap..... 3	11.10 Vector In Polygon..... 14	9f9204
4.4 Li Chao Tree..... 3	11.11 Polygon Cut..... 14	1 // CRYPTOGRAPHER
4.5 Time Segment Tree..... 3	11.12 Half-Plane Intersection..... 14	2 #define SZ(c) ((int)(c).size())
4.6 Segment Tree Beats..... 3	11.13 Cyclic Ternary Search..... 14	3 // kactl
5 DP 4	11.14 Circle Intersections..... 15	4 #define sz(x) (int)(x).size()
5.1 SOS DP..... 4	11.15 Tangents..... 15	5 #define rep(i,a,b) for(int i=(a);i<(b);i++)
5.2 Divide and Conquer DP..... 4	11.16 Bisector..... 15	6 using ll = long long;
5.3 Dynamic DP..... 4	11.17 Minimum Enclosing Circle..... 15	7 using vi = vector<int>;
5.4 Convex Hull Trick..... 4	11.18 Minkowski Sum..... 15	8 using pii = pair<int, int>;
6 Graph 4	11.19 Convex Hull Distance..... 15	9 // ckiseki
6.1 Max Clique..... 4	11.20 Closest Point Pair..... 15	10 #define all(x) begin(x), end(x)
6.2 Bellman-Ford..... 5	11.21 Polygon-Circle Intersection..... 16	11 #define pb emplace_back
6.3 System of Difference Constraints..... 5	11.22 Union of Polygons..... 16	12 #define REP(i, l, r) for (int i=(l); i<=(r); ++i)
6.4 Graph Girth..... 5	11.23 Rotating Sweep Line..... 16	13 const ll LINF = 1e18;
6.5 Euler Trail..... 5	11.24 Pick's Theorem..... 16	14 template <class A, class B> bool chmin(A& a, B& b) {
6.6 Vertex BCC (Round Square Tree)..... 6	12 Number Theory 16	15 return b < a ? a = b, 1 : 0;
6.7 Edge BCC..... 6	12.1 Mod Sum..... 16	16}
6.8 Kth Shortest Path..... 6	12.2 Extended Lucas Theorem..... 17	3.2 PBDS, Random
6.9 SCC - Tarjan..... 7	12.3 Prime Sieve and Defactor..... 17	d9e963
6.10 2SAT..... 7	12.4 Harmonic Series..... 17	1 #include <bits/extc++.h>
7 Tree 7	12.5 Count Number of Divisors..... 17	2 using namespace __gnu_pbds;
7.1 Tree Isomorphism (Rooted Trees)..... 7	12.6 數論分塊..... 17	3 // map
7.2 Tree Isomorphism (Unrooted	12.7 Pollard's rho..... 18	4 tree <int, int, less<>, rb_tree_tag,
Trees)..... 7	12.8 Miller Rabin..... 18	tree_order_statistics_node_update> tr;
7.3 Heavy Light Decomposition..... 7	12.9 Discrete Log..... 18	5 tr.order_of_key (element);
7.4 Virtual Tree..... 7	12.10 Discrete Sqrt..... 18	6 tr.find_by_order (rank);
8 Matching 8	12.11 Fast Power..... 18	7 // set
8.1 Bipartite Matching..... 8	12.12 Extend GCD..... 18	8 tree <int, null_type, less<>, rb_tree_tag,
8.2 Bipartite Weighted Matching..... 8	12.13 Mu + Phi..... 19	tree_order_statistics_node_update> tr;
8.3 General Matching..... 8	12.14 Other Formulas..... 19	9 tr.order_of_key (element);
9 Flow 9	12.15 Polynomial..... 19	10 tr.find_by_order (rank);
9.1 Flow Methods..... 9	12.16 Counting Primes..... 20	11 // priority queue
9.2 Dinic (Flow Decomposition)..... 9	12.17 Linear Sieve for Other Number	12 __gnu_pbds::priority_queue<int, less<int>> big_q;
9.3 ISAP..... 10	Theoretic Functions..... 21	13 __gnu_pbds::priority_queue<int, greater<int>> small_q;
9.4 Bounded Max Flow..... 10	13 Numerical 21	14 ql.join(q2); // join
9.5 MCMF..... 10	13.1 Polynomials and recurrences..... 21	15
9.6 Push-Relabel..... 11	13.2 Optimization..... 22	16 mt19937
9.7 Gomory-Hu Tree..... 11	13.3 Matrices..... 23	gen(chrono::steady_clock::now().time_since_epoch().count());
9.8 Global Min Cut..... 11	13.4 Fourier transforms..... 25	17 uniform_int_distribution <int>(a, b)(rng) // [a, b]
10 String 11	14 Combinatorics 25	18 uniform_real_distribution <double>(a, b)(rng) // (a, b)
10.1 Rolling Hash..... 11	14.1 Catalan Number..... 25	19 shuffle (v.begin(), v.end(), gen);
10.2 KMP..... 12	14.2 Bertrand's Ballot Theorem..... 25	3.3 Debug
10.3 Z Value..... 12	14.3 Burnside's Lemma..... 25	a265e1
10.4 Manacher..... 12	15 Special Numbers 25	1 #define debug(...) cerr << "\x1b[33m" #__VA_ARGS__ " :
10.5 Suffix Array + LCP..... 12	15.1 Fibonacci Series..... 25	2 \x1b[0m", \
10.6 Suffix Automaton..... 12	15.2 Prime Numbers..... 25	3 [](auto&&... a){ ((cerr << a << ' '), ...); }
10.7 Minimum Rotation..... 12	15.3 Number of Divisors..... 25	4 (__VA_ARGS__), cerr << '\n'
	15.4 Distinct Prime Factors..... 25	5 #define output(...) \
		6 [](auto&&... a){ ((cout << a << ' '), ...); }
		7 (__VA_ARGS__), cout << '\n'
		8 int v = 42;
		9 format ("{:b}", v); // "101010"
		10 format ("{:#b}", v); // "0b101010"
		11 format ("{:x}", v); // "0x2a"
		12 format ("{:#X}", v); // "0X2A"
		13 int x = 7;
		14 format ("{:05d}", x); // "00007"
		15 format ("{:>5}", x); // " 7"
		16 format ("{:<5}", x); // " 7 "
		17 format ("{:~5}", x); // " 7 "
		18 format ("{:~5}", x); // " 7 "
		19 format ("{:~5}", x); // " 7 "
		20 format ("{:~5}", x); // " 7 "
		21 format ("{:~5}", x); // " 7 "
		22 format ("{:~5}", x); // " 7 "
		23 format ("{:~5}", x); // " 7 "
		24 format ("{:~5}", x); // " 7 "
		25 format ("{:~5}", x); // " 7 "
		26 format ("{:~5}", x); // " 7 "
		27 format ("{:~5}", x); // " 7 "
		28 format ("{:~5}", x); // " 7 "
		29 format ("{:~5}", x); // " 7 "
		30 format ("{:~5}", x); // " 7 "
		31 format ("{:~5}", x); // " 7 "
		32 format ("{:~5}", x); // " 7 "
		33 format ("{:~5}", x); // " 7 "
		34 format ("{:~5}", x); // " 7 "
		35 format ("{:~5}", x); // " 7 "
		36 format ("{:~5}", x); // " 7 "
		37 format ("{:~5}", x); // " 7 "
		38 format ("{:~5}", x); // " 7 "
		39 format ("{:~5}", x); // " 7 "
		40 format ("{:~5}", x); // " 7 "
		41 format ("{:~5}", x); // " 7 "
		42 format ("{:~5}", x); // " 7 "
		43 format ("{:~5}", x); // " 7 "
		44 format ("{:~5}", x); // " 7 "
		45 format ("{:~5}", x); // " 7 "
		46 format ("{:~5}", x); // " 7 "
		47 format ("{:~5}", x); // " 7 "
		48 format ("{:~5}", x); // " 7 "
		49 format ("{:~5}", x); // " 7 "
		50 format ("{:~5}", x); // " 7 "
		51 format ("{:~5}", x); // " 7 "
		52 format ("{:~5}", x); // " 7 "
		53 format ("{:~5}", x); // " 7 "
		54 format ("{:~5}", x); // " 7 "
		55 format ("{:~5}", x); // " 7 "
		56 format ("{:~5}", x); // " 7 "
		57 format ("{:~5}", x); // " 7 "
		58 format ("{:~5}", x); // " 7 "
		59 format ("{:~5}", x); // " 7 "
		60 format ("{:~5}", x); // " 7 "
		61 format ("{:~5}", x); // " 7 "
		62 format ("{:~5}", x); // " 7 "
		63 format ("{:~5}", x); // " 7 "
		64 format ("{:~5}", x); // " 7 "
		65 format ("{:~5}", x); // " 7 "
		66 format ("{:~5}", x); // " 7 "
		67 format ("{:~5}", x); // " 7 "
		68 format ("{:~5}", x); // " 7 "
		69 format ("{:~5}", x); // " 7 "
		70 format ("{:~5}", x); // " 7 "
		71 format ("{:~5}", x); // " 7 "
		72 format ("{:~5}", x); // " 7 "
		73 format ("{:~5}", x); // " 7 "
		74 format ("{:~5}", x); // " 7 "
		75 format ("{:~5}", x); // " 7 "
		76 format ("{:~5}", x); // " 7 "
		77 format ("{:~5}", x); // " 7 "
		78 format ("{:~5}", x); // " 7 "
		79 format ("{:~5}", x); // " 7 "
		80 format ("{:~5}", x); // " 7 "
		81 format ("{:~5}", x); // " 7 "
		82 format ("{:~5}", x); // " 7 "
		83 format ("{:~5}", x); // " 7 "
		84 format ("{:~5}", x); // " 7 "
		85 format ("{:~5}", x); // " 7 "
		86 format ("{:~5}", x); // " 7 "
		87 format ("{:~5}", x); // " 7 "
		88 format ("{:~5}", x); // " 7 "
		89 format ("{:~5}", x); // " 7 "
		90 format ("{:~5}", x); // " 7 "
		91 format ("{:~5}", x); // " 7 "
		92 format ("{:~5}", x); // " 7 "
		93 format ("{:~5}", x); // " 7 "
		94 format ("{:~5}", x); // " 7 "
		95 format ("{:~5}", x); // " 7 "
		96 format ("{:~5}", x); // " 7 "
		97 format ("{:~5}", x); // " 7 "
		98 format ("{:~5}", x); // " 7 "
		99 format ("{:~5}", x); // " 7 "
		100 format ("{:~5}", x); // " 7 "
		101 format ("{:~5}", x); // " 7 "
		102 format ("{:~5}", x); // " 7 "
		103 format ("{:~5}", x); // " 7 "
		104 format ("{:~5}", x); // " 7 "
		105 format ("{:~5}", x); // " 7 "
		106 format ("{:~5}", x); // " 7 "
		107 format ("{:~5}", x); // " 7 "
		108 format ("{:~5}", x); // " 7 "
		109 format ("{:~5}", x); // " 7 "
		110 format ("{:~5}", x); // " 7 "
		111 format ("{:~5}", x); // " 7 "
		112 format ("{:~5}", x); // " 7 "
		113 format ("{:~5}", x); // " 7 "
		114 format ("{:~5}", x); // " 7 "
		115 format ("{:~5}", x); // " 7 "
		116 format ("{:~5}", x); // " 7 "
		117 format ("{:~5}", x); // " 7 "
		118 format ("{:~5}", x); // " 7 "
		119 format ("{:~5}", x); // " 7 "
		120 format ("{:~5}", x); // " 7 "
		121 format ("{:~5}", x); // " 7 "
		122 format ("{:~5}", x); // " 7 "
		123 format ("{:~5}", x); // " 7 "
		124 format ("{:~5}", x); // " 7 "
		125 format ("{:~5}", x); // " 7 "
		126 format ("{:~5}", x); // " 7 "
		127 format ("{:~5}", x); // " 7 "
		128 format ("{:~5}", x); // " 7 "
		129 format ("{:~5}", x); // " 7 "
		130 format ("{:~5}", x); // " 7 "
		131 format ("{:~5}", x); // " 7 "
		132 format ("{:~5}", x); // " 7 "
		133 format ("{:~5}", x); // " 7 "
		134 format ("{:~5}", x); // " 7 "
		135 format ("{:~5}", x); // " 7 "
		136 format ("{:~5}", x); // " 7 "
		137 format ("{:~5}", x); // " 7 "
		138 format ("{:~5}", x); // " 7 "
		139 format ("{:~5}", x); // " 7 "
		140 format ("{:~5}", x); // " 7 "
		141 format ("{:~5}", x); // " 7 "
		142 format ("{:~5}", x); // " 7 "
		143 format ("{:~5}", x); // " 7 "
		144 format ("{:~5}", x); // " 7 "
		145 format ("{:~5}", x); // " 7 "
		146 format ("{:~5}", x); // " 7 "
		147 format ("{:~5}", x); // " 7 "
		148 format ("{:~5}", x); // " 7 "
		149 format ("{:~5}", x); // " 7 "
		150 format ("{:~5}", x); // " 7 "
		151 format ("{:~5}", x); // " 7 "
		152 format ("{:~5}", x); // " 7 "
		153 format ("{:~5}", x); // " 7 "
		154 format ("{:~5}", x); // " 7 "
		155 format ("{:~5}", x); // " 7 "
		156 format ("{:~5}", x); // " 7 "
		157 format ("{:~5}", x); // " 7 "
		158 format ("{:~5}", x); // " 7 "
		159 format ("{:~5}", x); // " 7 "
		160 format ("{:~5}", x); // " 7 "
		161 format ("{:~5}", x); // " 7 "
		162 format ("{:~5}", x); // " 7 "
		163 format ("{:~5}", x); // " 7 "
		164 format ("{:~5}", x); // " 7 "
		165 format ("{:~5}", x); // " 7 "
		166 format ("{:~5}", x); // " 7 "
		167 format ("{:~5}", x); // " 7 "
		168 format ("{:~5}", x); // " 7 "
		169 format ("{:~5}", x); // " 7 "
		170 format ("{:~5}", x); // " 7 "
		171 format ("{:~5}", x); // " 7 "
		172 format ("{:~5}", x); // " 7 "
		173 format ("{:~5}", x); // " 7 "
		174 format ("{:~5}", x); // " 7 "
		175 format ("{:~5}", x); // " 7 "
		176 format ("{:~5}", x); // " 7 "
		177 format ("{:~5}", x); // " 7 "
		178 format ("{:~5}", x); // " 7 "
		179 format ("{:~5}", x); // " 7 "
		180 format ("{:~5}", x); // " 7 "
		181 format ("{:~5}", x); // " 7 "
		182 format ("{:~5}", x); // " 7 "
		183 format ("{:~5}", x); // " 7 "
		184 format ("{:~5}", x); // " 7 "
		185 format ("{:~5}", x); // " 7 "
		186 format ("{:~5}", x); // " 7 "
		187 format ("{:~5}", x); // " 7 "
		188 format ("{:~5}", x); // " 7 "
		189 format ("{:~5}", x); // " 7 "
		190 format ("{:~5}", x); // " 7 "
		191 format ("{:~5}", x); // " 7 "
		192 format ("{:~5}", x); // " 7 "
		193 format ("{:~5}", x); // " 7 "
		194 format ("{:~5}", x); // " 7 "
		195 format ("{:~5}", x); // " 7 "
		196 format ("{:~5}", x); // " 7 "
		197 format ("{:~5}", x); // " 7 "
		198 format ("{:~5}", x); // " 7 "
		199 format ("{:~5}", x); // " 7 "
		200 format ("{:~5}", x); // " 7 "
		201 format ("{:~5}", x); // " 7 "
		202 format ("{:~5}", x); // " 7 "
		203 format ("{:~5}", x); // " 7 "
		204 format ("{:~5}", x); // " 7 "
		205 format ("{:~5}", x); // " 7 "
		206 format ("{:~5}", x); // " 7 "
		207 format ("{:~5}", x); // " 7 "
		208 format ("{:~5}", x); // " 7 "
		209 format ("{:~5}", x); // " 7 "
		210 format ("{:~5}", x); // " 7 "
		211 format ("{:~5}", x); // " 7 "
		212 format ("{:~5}", x); // " 7 "
		213 format ("{:~5}", x); // " 7 "
		214 format ("{:~5}", x); // " 7 "
		215 format ("{:~5}", x); // " 7 "
		216 format ("{:~5}", x); // " 7 "
		217 format ("{:~5}", x); // " 7 "
		218 format ("{:~5}", x); // " 7 "
		219 format ("{:~5}", x); // " 7 "
		220 format ("{:~5}", x); // " 7 "
		221 format ("{:~5}", x); // " 7 "
		222 format ("{:~5}", x); // " 7 "
		223 format ("{:~5}", x); // " 7 "
		224 format ("{:~5}", x); // " 7 "
		225 format ("{:~5}", x); // " 7 "
		226 format ("{:~5}", x); // " 7 "
		227 format ("{:~5}", x); // " 7 "
		228 format ("{:~5}", x); // "

```

12     "viewBox='$ $ $ $>\n"
13     "<style>{*{stroke-width:0.5%;}</style>\n",
14     x1, -y2, x2 - x1, y2 - y1); }
15 ~SVG() { p("</svg>\n"); }
16 void color(string nc) { c = nc; }
17 void line(auto x1, auto y1, auto x2, auto y2) {
18     p("<line x1='$' y1='$' x2='$' y2='$' stroke='$' />\n",
19     x1, -y1, x2, -y2, c); }
20 void circle(auto x, auto y, auto r) {
21     p("<circle cx='$' cy='$' r='$' stroke='$' "
22     "fill='none' />\n", x, -y, r, c); }
23 void text(auto x, auto y, string s, int w = 12) {
24     p("<text x='$' y='$' font-size='$px'>$</text>\n",
25     x, -y, w, s); }
26 };

```

```

1 SVG svg("out.svg", -200, -200, 200, 200);
2 svg.color("lightgray"); // ↓ drawing x-y plane
3 svg.line(-200, 0, 200, 0); svg.line(0, -200, 0, 200);
4 svg.color("blue"); svg.line(10, 10, 80, 80);
5 svg.color("red"); svg.circle(50, 50, 1);

```

3.5 Python

00e632

```

1 import sys
2 input = sys.stdin.readline
3 readStr = lambda: input().rstrip('\r\n')
4 readInt = lambda: int(input())
5 readInts = lambda: list(map(int, input().split()))
6
7 from decimal import *
8 getcontext().prec = 50 # precision
9 x = Decimal(str(x))
10 y = Decimal("6.7")
11 x *= y
12 print(x.quantize(Decimal("0.000001"),
13     rounding=ROUND_HALF_EVEN))
14 # ROUND_CEILING : To INF (2.9 -> 3, -2.1 -> -2)
15 # ROUND_FLOOR : To -INF (2.1 -> 2, -2.9 -> -3)
16 # ROUND_HALF_EVEN: To nearest even (2.5 -> 2, 3.5 -> 4)
17 # ROUND_HALF_UP : Half away from 0 (2.5 -> 3, -2.5 -> -3)
18 # ROUND_HALF_DOWN: Half towards 0 (2.5 -> 2, -2.5 -> -2)
19 # ROUND_UP : Away from 0 (2.1 -> 3, -2.1 -> -3)
20 # ROUND_DOWN : Truncate to 0 (2.9 -> 2, -2.9 -> -2)
21 # <Default>: ROUND_HALF_EVEN
22 # <Standard School Math>: ROUND_HALF_UP

```

3.6 Stress Tests

```

1 from random import *
2 n = randint(1, 10)
3 arr = list(randint(1, 20) for i in range(n))
4 print(*arr)
5 # permutation / non-repetitive sequence
6 perm = sample(range(1, n + 1), n)
7 arr = sample(range(1, 10000 + 1), n)
8 # string
9 from string import *
10 s = "".join(choices(ascii_lowercase, k=n))
11 t = "".join(choices(ascii_lowercase[:3], k=n)) # (a|b|c)*
12 u = "".join(choices(ascii_lowercase + ascii_uppercase +
13     digits, k=n))
14 # palindrome
15 half = "".join(choices(ascii_lowercase, k=(n + 1) // 2))
16 palindrome = half + (half[-2::-1] if n % 2 else half[::-1])
17 # ===== tree =====
18 n = 30
19 ## shallow tree
20 edges = [(randint(1, u - 1), u) for u in range(2, n + 1)]
21 ## deep tree
22 ## (suggested k: 2 ~ 5 or int(sqrt(n)) or 2n/D (D is
23     expected depth))
24 from math import *
25 k = int(sqrt(n))
26 edges = [(randint(max(1, u-k), u-1), u) for u in range(2, n
27     + 1)]
28 for e in edges:
29     print(*e)
30
31 edges = [(1, i) for i in range(2, n + 1)] ## star
32 edges = [(i, i + 1) for i in range(1, n)] ## chain
33 edges = [(i // 2, i) for i in range(2, n + 1)] ## complete
34     binary tree
35 # ===== graph =====
36 ## not necessarily connected, no multi-edge or self-loop
37 n = 10
38 density = 0.3 # graph density (0.0 ~ 1.0)
39 edges = [(i, j) for i in range(1, n+1) for j in range(i+1,
40     n+1) if random() < density]
41 ## connected, no multi-edge or self-loop
42 n, extra_edges = 10, 5
43 edges = set()
44 ##### ... generate spanning tree with codes above

```

```

40 while len(edges) < n - 1 + extra_edges:
41     u, v = randint(1, n), randint(1, n)
42     if u != v:
43         edges.add((min(u, v), max(u, v)))
44 edges = list(edges)
45 ## DAG
46 perm = sample(range(1, n + 1), n)
47 edges = []
48 for _ in range(m):
49     i, j = sorted(sample(range(n), 2))
50     edges.append((perm[i], perm[j]))
51 ## Bipartite Graph
52 n1, n2 = 6, 7
53 left, right = list(range(1, n1+1)), list(range(n1+1,
54     n1+n2+1))
55 edges = [(choice(left), choice(right)) for _ in range(m)]

```

```

1 g++ -std=c++20 -o buggy $1
2 g++ -std=c++20 -o ac ac.cpp
3 for i in {1..100}; do

```

```

4     echo "TEST $i"
5     python3 gen.py > in
6     ./ac < in > oa
7     ./buggy < in > ob
8     if ! diff -q oa ob; then
9         cat in <(echo "=== AC") oa <(echo "=== WA") ob
10        break
11    fi
12 done

```

4 Data Structure

4.1 Mo's Algorithm

8d7194

```

1 // segments are 0-based
2 int B = max(1, (int)sqrt(n));
3 sort(Q.begin(), Q.end(), [&](const auto& a, const auto& b) {
4     if (a[0]/B != b[0]/B) return a[0]/B < b[0]/B;
5     return (a[0]/B & 1) ? a[1] < b[1] : a[1] > b[1];
6 });
7 ll cur = 0; // current answer
8 int pl = 0, pr = -1;
9 for (auto& qi : Q) {
10    // get (l, r, qid) from qi
11    while (pl < l) del(pl++); while (pl > l) add(--pl);
12    while (pr < r) add(++pr); while (pr > r) del(pr--);
13    ans[qid] = cur;
14 }

```

4.2 CDQ

744a80

```

1 // preparation
2 // 1. write a 1-base BIT
3 // 2. init(n, mxc): n is number of elements, mxc = 值域
4 // 3. build info array CDQ.v by yourself (x, y, z, id)
5 // 4. call solve()
6 // => cdq.ans[i] is number of j s.t. (xj <= xi, yj <= yi, zj
7     <= zi) (j != i)
8 struct CDQ {
9     struct info {
10         int x, y, z, id;
11         info(int x, int y, int z, int id):
12             x(x), y(y), z(z), id(id) {}
13         info():
14             x(0), y(0), z(0), id(0) {}
15     };
16     int n, mxc;
17     BIT bit;
18     vector<info> v;
19     vector<ll> ans;
20     void init(int _n, int _mxc) {
21         n = _n; mxc = _mxc;
22         v.assign(n, info{});
23         ans.assign(n, 0);
24     }
25     void dfs(int l, int r) {
26         if (l == r) return;
27         int mid = (l+r) >> 1;
28         dfs(l, mid); dfs(mid+1, r);
29
30         vector<info> tmp;
31         vector<int> bit_op;
32         int pl = l, pr = mid+1;
33
34         while (pl <= mid && pr <= r) {
35             if (v[pl].y <= v[pr].y) {
36                 tmp.emplace_back(v[pl]);
37                 bit.upd(v[pl].z, 1);
38                 bit_op.emplace_back(v[pl].z);
39                 pl++;
40             }
41             else {

```

```

41     tmp.emplace_back(v[pr]);
42     ans[v[pr].id] += bit.qry(v[pr].z);
43     pr++;
44 }
45 }
46 while (pl <= mid) {
47     tmp.emplace_back(v[pl]);
48     pl++;
49 }
50 while (pr <= r) {
51     tmp.emplace_back(v[pr]);
52     ans[v[pr].id] += bit.qry(v[pr].z);
53     pr++;
54 }
55 for (int i = l, j = 0; i <= r; i++, j++) v[i] = tmp[j];
56 for (auto& op : bit_op) bit.upd(op, -1);
57 }
58 void solve() {
59     bit.init(mxc);
60     sort(v.begin(), v.end(), [&](const info& a, const info&
61 b){
62         return (a.x == b.x ? (a.y == b.y ? (a.z == b.z ?
63 a.id < b.id : a.z < b.z) : a.y < b.y) : a.x < b.x);
64 });
65     dfs(0, n-1);
66     map<pair<int, pii>, int> s;
67     sort(v.begin(), v.end(), [&](const info& a, const info&
68 b){
69         return a.id > b.id;
70 });
71     for (int i = 0, id = n-1; i < n; i++, id--) {
72         pair<int, pii> tmp = make_pair(v[i].x,
73 make_pair(v[i].y, v[i].z));
74         auto it = s.find(tmp);
75         if (it != s.end())
76             ans[id] += it->second;
77         s[tmp]++;
78     }
79 }
80 }

```

4.3 Persistent Treap

803cce

```

1// Author: Ian
2struct node {
3    node *l, *r;
4    char c; int v, sz;
5    node(char x = '$'): c(x), v(mt()), sz(1) {
6        l = r = nullptr;
7    }
8    node(node* p) { *this = *p; }
9    void pull() {
10        sz = 1;
11        for (auto i : {l, r})
12            if (i) sz += i->sz;
13    }
14} arr[maxn], *ptr = arr;
15inline int size(node* p) {return p ? p->sz : 0;}
16node* merge(node* a, node* b) {
17    if (!a || !b) return a ? : b;
18    if (a->v < b->v) {
19        node* ret = new(ptr++) node(a);
20        ret->r = merge(ret->r, b); ret->pull();
21        return ret;
22    }
23    else {
24        node* ret = new(ptr++) node(b);
25        ret->l = merge(a, ret->l); ret->pull();
26        return ret;
27    }
28}
29P<node*> split(node* p, int k) {
30    if (!p) return {nullptr, nullptr};
31    if (k >= size(p->l) + 1) {
32        auto [a, b] = split(p->r, k - size(p->l) - 1);
33        node* ret = new(ptr++) node(p);
34        ret->r = a, ret->pull();
35        return {ret, b};
36    }
37    else {
38        auto [a, b] = split(p->l, k);
39        node* ret = new(ptr++) node(p);
40        ret->l = b, ret->pull();
41        return {a, ret};
42    }
43}

```

4.4 Li Chao Tree

6c33c2

```

1// Author: Ian
2// Function: For a set of lines L, find the maximum L_i(x)
in L in O(lg n).

```

```

3typedef long double ld;
4constexpr int maxn = 5e4 + 5;
5struct line {
6    ld a, b;
7    ld operator()(ld x) {return a * x + b;}
8} arr[(maxn + 1) << 2];
9bool operator<(line a, line b) {return a.a < b.a;}
10#define m ((l+r)>>1)
11void insert(line x, int i = 1, int l = 0, int r = maxn) {
12    if (r - l == 1) {
13        if (x(l) > arr[i](l))
14            arr[i] = x;
15        return;
16    }
17    line a = max(arr[i], x), b = min(arr[i], x);
18    if (a(m) > b(m))
19        arr[i] = a, insert(b, i << 1, l, m);
20    else
21        arr[i] = b, insert(a, i << 1 | 1, m, r);
22}
23ld query(int x, int i = 1, int l = 0, int r = maxn) {
24    if (x < l || r <= x) return -numeric_limits<ld>::max();
25    if (r - l == 1) return arr[i](x);
26    return max({arr[i](x), query(x, i << 1, l, m), query(x, i
<< 1 | 1, m, r)});
27}
28#undef m

```

4.5 Time Segment Tree

519775

```

1// Author: Ian
2struct Op; // reversible operation
3const int maxn = 2e5 + 5;
4vector<Op> arr[(maxn + 1) << 2];
5#define m ((l+r)>>1)
6void insert(int ql, int qr, Op x, int i=1, int l=0, int r=maxn) {
7    if (qr <= l || r <= ql) return;
8    if (ql <= l && r <= qr) { arr[i].push_back(x); return; }
9    if (qr <= m) insert(ql, qr, x, i << 1, l, m);
10    else if (m <= ql) insert(ql, qr, x, i << 1 | 1, m, r);
11    else {
12        insert(ql, qr, x, i << 1, l, m);
13        insert(ql, qr, x, i << 1 | 1, m, r);
14    }
15}
16void traversal(vector<int>& ans, int i=1, int l=0, int r=maxn){
17    int opcnt = 0;
18    for (auto val : arr[i])
19        if (DO_SOMETHING(val)) opcnt++;
20    if (r - l == 1) ans[l] = CALC_ANS();
21    else {
22        traversal(ans, i << 1, l, m);
23        traversal(ans, i << 1 | 1, m, r);
24    }
25    while (opcnt--) UNDO_SOMETHING();
26    arr[i].clear();
27}
28#undef m

```

4.6 Segment Tree Beats

f686aa

```

1// Author: Ian, https://judge.yosupo.jp/submission/400861
2const int maxn=2e5+5;
3const ll MX = 1e18+5;
4const ll MN = -MX;
5#define m ((l+r)>>1)
6struct node {
7    ll sum, inc;
8    ll mn, smn; int mnc;
9    ll mx, smx; int mxc;
10    node(ll x=0): sum(x), inc(0),
11        mn(x), smn(MN), mnc(1),
12        mx(x), smx(MN), mxc(1) {}
13    void pull(int i);
14    void push(int i, int l, int r);
15} a[(maxn+1)<<2];
16void node::pull(int i) {
17    sum = a[i<<1].sum+a[i<<1|1].sum;
18    if (a[i<<1].mn == a[i<<1|1].mn) {
19        mn = a[i<<1].mn;
20        smn = min(a[i<<1].smn, a[i<<1|1].smn);
21        mnc = a[i<<1].mnc+a[i<<1|1].mnc;
22    } else {
23        if (a[i<<1].mn < a[i<<1|1].mn) {
24            mn = a[i<<1].mn;
25            smn = min(a[i<<1].smn, a[i<<1|1].smn);
26            mnc = a[i<<1].mnc;
27        } else {
28            mn = a[i<<1|1].mn;
29            smn = min(a[i<<1|1].smn, a[i<<1].smn);
30            mnc = a[i<<1|1].mnc;
31        }
32    }
33    if (a[i<<1].mx == a[i<<1|1].mx) {

```

```

33 mx = a[i<<1].mx;
34 smx = max(a[i<<1].smx, a[i<<1|1].smx);
35 mxc = a[i<<1].mxc+a[i<<1|1].mxc;
36 } else {
37 if (a[i<<1].mx > a[i<<1|1].mx) {
38 mx = a[i<<1].mx;
39 smx = max(a[i<<1].smx, a[i<<1|1].mx);
40 mxc = a[i<<1].mxc;
41 } else {
42 mx = a[i<<1|1].mx;
43 smx = max(a[i<<1|1].smx, a[i<<1].mx);
44 mxc = a[i<<1|1].mxc;
45 } }
46 }
47 void updmn(int i, int len, ll v) {
48 a[i].sum += (v-a[i].mn)*a[i].mnc; a[i].mn = v;
49 if (len==1||v>=a[i].mx)
50 a[i].mx = v, a[i].smx = MN;
51 else if (v>a[i].smx)
52 a[i].smx = v;
53 }
54 void updmx(int i, int len, ll v) {
55 a[i].sum -= (a[i].mx-v)*a[i].mxc; a[i].mx = v;
56 if (len==1||v<=a[i].mn)
57 a[i].mn = v, a[i].smn = MX;
58 else if (v<a[i].smn)
59 a[i].smn = v;
60 }
61 void node::push(int i, int l, int r) {
62 if (r-l==1) return;
63 pair<int, int> ch[] = {{i<<1, m-l}, {i<<1|1, r-m}};
64 if (inc!=0) for (auto [j, len]: ch) {
65 a[j].sum += inc*len, a[j].inc += inc;
66 a[j].mx += inc, a[j].smx += inc;
67 a[j].mn += inc, a[j].smn += inc;
68 } inc = 0;
69 for (auto [j, len]: ch) {
70 if (a[j].mn < mn) updmn(j, len, mn);
71 if (a[j].mx > mx) updmx(j, len, mx); }
72 }
73 void build(vector<ll>& v, int i=1, int l=0, int r=maxn) {
74 if ((int)v.size()<=l) return;
75 if (r-l==1) { a[i] = node(v[l]); return; }
76 build(v, i<<1, l, m);
77 build(v, i<<1|1, m, r);
78 a[i].pull(i);
79 }
80 void chmin(int ql, int qr, ll v, int i=1, int l=0, int r=maxn) {
81 if (qr<=l||r<=ql||a[i].mx<=v) return;
82 if (ql<=l&&r<=qr&&a[i].smx<v) {updmx(i, r-l, v);return;}
83 a[i].push(i, l, r);
84 chmin(ql, qr, v, i<<1, l, m);
85 chmin(ql, qr, v, i<<1|1, m, r);
86 a[i].pull(i);
87 }
88 void chmax(int ql, int qr, ll v, int i=1, int l=0, int r=maxn) {
89 if (qr<=l||r<=ql||a[i].mn>=v) return;
90 if (ql<=l&&r<=qr&&a[i].smn>v) {updmn(i, r-l, v);return;}
91 a[i].push(i, l, r);
92 chmax(ql, qr, v, i<<1, l, m);
93 chmax(ql, qr, v, i<<1|1, m, r);
94 a[i].pull(i);
95 }
96 void inc(int ql, int qr, ll v, int i=1, int l=0, int r=maxn) {
97 if (qr<=l || r<=ql) return;
98 if (ql<=l && r<=qr) {
99 a[i].sum += v*(r-l);
100 a[i].mx += v, a[i].smx += v;
101 a[i].mn += v, a[i].smn += v;
102 a[i].inc += v;
103 return;
104 }
105 a[i].push(i, l, r);
106 inc(ql, qr, v, i<<1, l, m);
107 inc(ql, qr, v, i<<1|1, m, r);
108 a[i].pull(i);
109 }
110 ll qsum(int ql, int qr, int i=1, int l=0, int r=maxn) {
111 if (qr<=l || r<=ql) return 0;
112 if (ql<=l && r<=qr) return a[i].sum;
113 a[i].push(i, l, r);
114 return qsum(ql, qr, i<<1, l, m)+qsum(ql, qr, i<<1|1, m, r);
115 }
116 #undef m

```

5 DP

5.1 SOS DP

623a08

// Author: Gino

```

2// Function: Solve problems that enumerates subsets of
subsets (3^n => n*2^n)
3for (int msk = 0; msk < (1<<n); msk++) {
4for (int i = 1; i <= n; i++) {
5if (msk & (1<<(i - 1))) {
6// dp[msk][i] = dp[msk][i - 1] + dp[msk ^ (1<<(i - 1))][i -
1];
7} else {
8// dp[msk][i] = dp[msk][i - 1];
9} } }

```

5.2 Divide and Conquer DP

d36c77

```

1/* Author: Simon Lindholm (License: CC0)
2 * Description: Given $a[i] = \min_{f(i, k)} \{ \le k < hi(i) \}
(f(i, k))$ where the (minimal)
3 * optimal $k$ increases with $i$, computes $a[i]$ for $i =
L..R-1$.
4 * Time: $O((N + (hi-lo)) \log N)$ */
5struct DP { // Modify at will:
6int lo(int ind) { return 0; }
7int hi(int ind) { return ind; }
8ll f(int ind, int k) { return dp[ind][k]; }
9void store(int ind, int k, ll v) { res[ind] = pii(k, v); }
10
11void rec(int L, int R, int LO, int HI) {
12if (L >= R) return;
13int mid = (L + R) >> 1;
14pair<ll, int> best(LLONG_MAX, LO);
15rep(k, max(LO.lo(mid), min(HI.hi(mid))))
16best = min(best, make_pair(f(mid, k), k));
17store(mid, best.second, best.first);
18rec(L, mid, LO, best.second+1);
19rec(mid+1, R, best.second, HI);
20}
21void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
22};

```

5.3 Dynamic DP

- 適用情境： $dp_i = M_i \cdot dp_{i-1} \Rightarrow dp_i = M_i M_{i-1} \dots M_1 dp_0$
- 當 M_i 需要動態修改，且 M_i 是廣義矩陣乘法：

$$C_{ij} = \bigoplus_k A_{ik} \otimes B_{kj}$$

- 滿足 $(\oplus, \otimes)$ 是半環 ($\otimes$ 對 $\oplus$ 有分配律、 $\otimes$ 和 $\oplus$ 有結合律、 $\oplus$ 有交換律)
- 此時便可用線段樹維護 $M_n M_{n-1} \dots M_1$ 的乘積，線段樹上的 pos 是 M_{pos} ，但 pull 時要 $st[idx] = mul(st[cr], st[cl])$

5.4 Convex Hull Trick

bb4a1c

```

1/**
2 * Author: Simon Lindholm (License: CC0, Source: own work)
3 * Container where you can add lines of the form kx+m,
4 * and query maximum values at points x.
5 * Useful for dynamic programming ('convex hull trick').
6 * Time: $O(\log N)$, Status: stress-tested */
7struct Line {
8mutable ll k, m, p;
9bool operator<(const Line& o) const { return k < o.k; }
10bool operator<(ll x) const { return p < x; }
11};
12struct LineContainer : multiset<Line, less<>> {
13// (for doubles, use inf = 1/.0, div(a,b) = a/b)
14static const ll inf = LLONG_MAX;
15ll div(ll a, ll b) { // floored division
16return a / b - ((a ^ b) < 0 && a % b); }
17bool isect(iterator x, iterator y) {
18if (y == end()) return x->p = inf, 0;
19if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
20else x->p = div(y->m - x->m, x->k - y->k);
21return x->p >= y->p;
22}
23void add(ll k, ll m) {
24auto z = insert({k, m, 0}), y = z++, x = y;
25while (isect(y, z)) z = erase(z);
26if (x != begin() && isect(--x, y)) isect(x, y =
erase(y));
27while ((y = x) != begin() && (--x)->p >= y->p)
isect(x, erase(y));
28}
29}
30ll query(ll x) {
31auto l = *lower_bound(x);
32return l.k * x + l.m;
33};

```

6 Graph

6.1 Max Clique

3a1fe5

```

1// max N = 64
2typedef unsigned long long ll;

```

```

3 struct MaxClique{
4     ll nb[ N ] , n , ans;
5     void init( ll _n ){
6         n = _n;
7         for( int i = 0 ; i < n ; i ++ ) nb[ i ] = 0LLU;
8     }
9     void add_edge( ll _u , ll _v ){
10        nb[ _u ] |= ( 1LLU << _v );
11        nb[ _v ] |= ( 1LLU << _u );
12    }
13    void B( ll r , ll p , ll x , ll cnt , ll res ){
14        if( cnt + res < ans ) return;
15        if( p == 0LLU && x == 0LLU ){
16            if( cnt > ans ) ans = cnt;
17            return;
18        }
19        ll y = p | x; y &= -y;
20        ll q = p & ( ~nb[ int( log2( y ) ) ] );
21        while( q ){
22            ll i = int( log2( q & (-q) ) );
23            B( r | ( 1LLU << i ) , p & nb[ i ] , x & nb[ i ]
24            nb[ i ] ) , cnt + 1LLU , __builtin_popcountll( p &
25            nb[ i ] ) );
26            q &= ~( 1LLU << i );
27            p &= ~( 1LLU << i );
28            x |= ( 1LLU << i );
29        }
30    }
31    int solve(){
32        ans = 0;
33        ll _set = 0;
34        if( n < 64 ) _set = ( 1LLU << n ) - 1;
35        else{
36            for( ll i = 0 ; i < n ; i ++ ) _set |= ( 1LLU << i );
37        }
38        B( 0LLU , _set , 0LLU , 0LLU , n );
39        return ans;
40    }
41 }maxClique;

```

6.2 Bellman-Ford

92b21f

```

1 // Author: Gino
2 // n, m = #(vertices), #(edges), and vertices are 1-based
3 // 1. BellmanFord bf(G, n, m);
4 // 2-1: bf.calcDis(s);
5 // => bf.dis[u] := shortest dis from s to u
6 // => bf.dis[u] := LINF (s can't reach u)
7 // => bf.dis[u] := -LINF (dis[u] can be arbitrary small)
8 // 2-2: bf.findNegCycle();
9 // => false (if no neg cycle)
10 // => true (has neg cycle & gives an example: bf.neg_cycle)
11 // Time: O(VE), Space: O(V + E)
12 const ll LINF = 4e18;
13 struct BellmanFord {
14     const vector<vector<pair<int, ll>>& G;
15     int n, m; // #(vertices), #(edges)
16     BellmanFord(const auto& G, int n, int m):G(G), n(n), m(m) {}
17     vector<ll> dis; vector<int> nth_relax, pa;
18     void run_bf(const auto& src) {
19         dis.assign(n+1, LINF);
20         nth_relax.assign(n+1, 0); pa.assign(n+1, -1);
21         for (auto& s : src) dis[s] = 0;
22         for (int rlx = 1; rlx <= n; rlx++) {
23             for (int u = 1; u <= n; u++) {
24                 if (dis[u] == LINF) continue; // !important
25                 for (auto& [v, w] : G[u]) {
26                     if (dis[v] > dis[u] + w) {
27                         dis[v] = dis[u] + w; pa[v] = u;
28                         if (rlx == n) nth_relax[v] = 1;
29                     }
30                 }
31             }
32         }
33         // 5 brackets
34         void calcDis(int s) {
35             run_bf(vector<int>{s}); queue<int> q;
36             for (int u=1; u<=n; u++) if (nth_relax[u]) q.push(u);
37             while (!q.empty()) { int u = q.front(); q.pop();
38                 for (auto& [v, w] : G[u]) {
39                     if (!nth_relax[v]) {
40                         nth_relax[v] = 1; q.push(v);
41                     }
42                 }
43             }
44             for (int u=1; u<=n; u++) if (nth_relax[u]) dis[u] = -LINF;
45         }
46     }
47     vector<int> neg_cycle;
48     bool findNegCycle() {
49         auto src = views::iota(1, n + 1); run_bf(src);
50         auto it = ranges::find_if(src,
51             [&](int s){ return nth_relax[s]; });
52         if (it == src.end()) return false;
53         int ptr = *it; for (int i=0; i<n; i++) ptr = pa[ptr];
54         neg_cycle.clear(); int cur = ptr;
55         while (true) {

```

```

49         neg_cycle.emplace_back(cur);
50         if (cur == ptr && neg_cycle.size() > 1) break;
51         cur = pa[cur];
52     }
53     ranges::reverse(neg_cycle); return true;
54 }

```

6.3 System of Difference Constraints

fe55f9

```

1 vector<vector<pair<int, ll>>> G;
2 void add(int u, int v, ll w) { G[u].push_back({v, w}); }

```

- $x_u - x_v \leq c \Rightarrow \text{add}(v, u, c)$
- $x_u - x_v \geq c \Rightarrow \text{add}(u, v, -c)$
- $x_u - x_v = c \Rightarrow \text{add}(v, u, c), \text{add}(u, v, -c)$
- $x_u \geq c \Rightarrow \text{add super vertex } x_0 = 0, \text{ then } x_u - x_0 \geq c \Rightarrow \text{add}(u, 0, -c)$
- Don't forget non-negative constraints for every variable if specified implicitly.
- Interval sum $\Rightarrow$ Use prefix sum to transform into differential constraints. Don't forget $S_{i+1} - S_i \geq 0$ if x_i needs to be non-negative.
- $x_u/x_v \leq c \Rightarrow \log x_u - \log x_v \leq \log c$

6.4 Graph Girth

Run BFS for every node, when encountered non-BFS-tree edge, update answer (min cycle length) with $\text{dis}[u] + \text{dis}[v] + 1$. Time $O(VE)$.

6.5 Euler Trail

11c393

```

1 // Author: kactl (Simon Lindholm), wrapped by Gino
2 // Vertices are 1-based, Edge ID are 0-based
3 // 1. EulerWalk euler(n, true(directed) / false(undirected));
4 // 2. euler.addEdge(u, v);
5 // 3. int result = euler.solve();
6 // => -1: nothing (euler.path, euler.path_edges = {})
7 // => 1: Euler trail,
8 // => 2: Euler circuit
9 //     euler.path = {u_1, ..., u_{m+1}}
10 //     euler.path_edges = {e_1, ..., e_m}
11 // Time: O(V + E), Space: O(V + E)
12 // Test: V, E <= 2e5, 99ms
13 struct EulerWalk {
14     int n, m = 0; bool dir;
15     vector<vector<pii>> G; vi deg;
16     EulerWalk(int n, bool dir) :
17         n(n), dir(dir), G(n + 1), deg(n + 1) {}
18     void addEdge(int u, int v) {
19         G[u].push_back({v, m});
20         if (!dir) G[v].push_back({u, m});
21         deg[u]++, deg[v] += dir ? -1 : 1; m++;
22     }
23     vi path, path_edges;
24     void walk(int src) {
25         path.clear(); path_edges.clear();
26         vi its(n + 1), visE(m);
27         function<void(int, int)> dfs = [&](int u, int e_in) {
28             while (its[u] < sz(G[u])) {
29                 auto [v, e] = G[u][its[u]++];
30                 if (visE[e]) continue;
31                 visE[e] = 1;
32                 dfs(v, e);
33             }
34             path.push_back(u);
35             if (e_in != -1) path_edges.push_back(e_in);
36         };
37         dfs(src, -1);
38         if (sz(path) != m + 1) {
39             path.clear(); path_edges.clear(); return;
40         }
41         ranges::reverse(path); ranges::reverse(path_edges);
42     }
43     int solve() {
44         int s = -1, odd = 0, src = 0, dst = 0;
45         for (int u = 1; u <= n; u++) {
46             if (!dir && (deg[u] & 1)) odd++, s = u;
47             if (dir && deg[u] == 1) s = u, src++;
48             if (dir && deg[u] == -1) dst++;
49             if (dir && abs(deg[u]) > 1) return -1;
50         }
51         if (!dir && odd != 0 && odd != 2) return -1;
52         if (dir && !((src == 0 && dst == 0) || (src == 1 && dst == 1))) return -1;
53         if (s == -1) for (int u = 1; u <= n; u++)
54             if (!G[u].empty()) { s = u; break; }
55         if (s == -1) s = 1;
56         walk(s);
57         return path.empty() ? -1 :
58             path.front() == path.back() ? 2 : 1;
59     }
60 };

```

6.6 Vertex BCC (Round Square Tree)

e85b0a

```

1 // Author: Ian, https://judge.yosupo.jp/submission/399308
2 // Vertex BCC (Round Square Tree) 0-based; Edge ID 0-based
3 // 1. BCC bcc(n); // n = #(vertices)
4 // 2. bcc.add_edge(u, v);
5 // 3. bcc.build();
6 // => bcc[id]: vertices in bcc 'id' (square node n+id in T)
7 // => T[id]: tree adjacency; square nodes are n..n+ncnt-1
8 // => is_cut(x): whether x is a cut vertex
9 // => bcc_id(u, v): square node containing tree-edge (u, v)
10 struct BCC {
11     int n, m, t, ncnt;
12     vector<vector<pii>> G;
13     vector<vector<int>> T, bcc;
14     vector<int> dep, low, pa2, dep2, s;
15     void add_edge(int u, int v) {
16         G[u].eb(v, m), G[v].eb(u, m++);
17     }
18     void dfs(int x, int f) {
19         if (~dep[x]) return;
20         dep[x] = low[x] = t++;
21         s.push_back(x);
22         for (auto [y, e]: G[x]) if (e != f) {
23             if (dep[y] == -1) {
24                 dfs(y, e), low[x] = min(low[x], low[y]);
25                 if (low[y] >= dep[x]) {
26                     int id = n + ncnt++; p;
27                     bcc.resize(bcc.size()+1);
28                     do {
29                         p = s.back(), s.pop_back();
30                         T[id].push_back(p), T[p].push_back(id);
31                         bcc.back().push_back(p);
32                     } while (p != y);
33                     T[id].push_back(x), T[x].push_back(id);
34                     bcc.back().push_back(x);
35                 } else low[x] = min(low[x], dep[y]);
36             }
37         }
38         void dfs2(int x, int p) {
39             dep2[x] = ~p ? dep2[p] + 1 : 0, pa2[x] = p;
40             for (auto y: T[x]) if (y != p) dfs2(y, x);
41         }
42         bool is_cut(int x) { return (int)T[x].size() != 1; }
43         int bcc_id(int u, int v) {
44             return pa2[dep2[u] < dep2[v] ? v : u] - n;
45         }
46         void build() {
47             for (int i = 0; i < n; i++) if (dep[i] == -1) {
48                 if (G[i].empty()) { // isolated vertex: singleton bcc
49                     dep[i] = low[i] = t++;
50                     int id = n + ncnt++;
51                     bcc.push_back({i});
52                     T[id].push_back(i), T[i].push_back(id);
53                 } else dfs(i, -1);
54             }
55             dfs2(i, -1);
56         }
57     }
58     BCC(int n): n(n), m(0), t(0), ncnt(0), G(n), T(2*n),
59     dep(n, -1), low(n), pa2(2*n), dep2(2*n) {}
60 };

```

6.7 Edge BCC

26729d

```

1 // Author: Ian, https://judge.yosupo.jp/submission/399295
2 // Edge BCC, 0-based; Edge ID 0-based
3 // 1. EBCC ebcc(n); // n = #(vertices)
4 // 2. ebcc.add_edge(u, v);
5 // 3. ebcc.build();
6 // => bcc[i]: each edge bcc
7 // => bccn[x]: in which bcc is x
8 // => isbr[e]: whether edge e is a bridge
9 struct EBCC {
10     int n, m, t, ncnt;
11     vector<vector<pii>> G;
12     vector<vector<int>> bcc;
13     vector<bool> isbr;
14     vector<int> dep, low, bccn, s;
15     EBCC(int n): n(n), m(0), t(0), ncnt(0), G(n),
16     dep(n, -1), low(n), bccn(n) {}
17     void add_edge(int u, int v) {
18         G[u].eb(v, m), G[v].eb(u, m++);
19     }
20     void dfs(int x, int f) {
21         if (~dep[x]) return;
22         dep[x] = low[x] = t++;
23         s.push_back(x);
24         for (auto [y, e]: G[x]) if (e != f) {
25             if (dep[y] == -1) {
26                 dfs(y, e), low[x] = min(low[x], low[y]);
27             } else low[x] = min(low[x], dep[y]);
28         }
29         if (dep[x] == low[x]) {
30             if (~f) isbr[f] = true;
31             bcc.resize(bcc.size()+1);
32             int p; do {

```

```

33         bccn[p = s.back()] = ncnt;
34         s.pop_back();
35         bcc.back().push_back(p);
36         while (p != x); ncnt++;
37     }
38     void build() {
39         isbr.assign(m, false);
40         for (int i=0; i<n; i++) dfs(i, -1);
41     }

```

6.8 Kth Shortest Path

34bacd

```

1 // time: O(|E| lg |E|+|V| lg |V|+K)
2 // memory: O(|E| lg |E|+|V|)
3 struct KSP { // 1-base
4     struct nd {
5         int u, v; ll d;
6         nd(int ui=0, int vi=0, ll di=INF) { u=ui; v=vi; d=di; }
7     };
8     struct heap { nd* edge; int dep; heap* chd[4]; };
9     static int cmp(heap* a, heap* b) {
10         return a->edge->d > b->edge->d;
11     }
12     struct node {
13         int v; ll d; heap* H; nd* E;
14         node(ll _d, int _v, nd* _E) { d=_d; v=_v; E=_E; }
15         node(heap* _H, ll _d) { H=_H; d=_d; }
16         friend bool operator<(node a, node b) {
17             return a.d > b.d;
18         }
19     };
20     int n, k, s, t, dst[N]; nd *nxt[N];
21     vector<nd*> g[N], rg[N]; heap *nullNd, *head[N];
22     void init(int _n, int _k, int _s, int _t) {
23         n=_n; k=_k; s=_s; t=_t;
24         for (int i=1; i<=n; i++) {
25             g[i].clear(); rg[i].clear();
26             nxt[i]=NULL; head[i]=NULL; dst[i]=-1;
27         }
28     }
29     void addEdge(int ui, int vi, ll di) {
30         nd* e=new nd(ui, vi, di);
31         g[ui].push_back(e); rg[vi].push_back(e);
32     }
33     queue<int> dfsQ;
34     void dijkstra() {
35         while (dfsQ.size()) dfsQ.pop();
36         priority_queue<node> Q; Q.push(node(0, t, NULL));
37         while (!Q.empty()) {
38             node p=Q.top(); Q.pop(); if (dst[p.v] != -1) continue;
39             dst[p.v]=p.d; nxt[p.v]=p.E; dfsQ.push(p.v);
40             for (auto e: rg[p.v]) Q.push(node(p.d+e->d, e->u, e));
41         }
42     }
43     heap* merge(heap* curNd, heap* newNd) {
44         if (curNd==nullNd) return newNd;
45         heap* root=new heap; memcpy(root, curNd, sizeof(heap));
46         if (newNd->edge->d < curNd->edge->d) {
47             root->edge=newNd->edge;
48             root->chd[2]=newNd->chd[2];
49             root->chd[3]=newNd->chd[3];
50             newNd->edge=curNd->edge;
51             newNd->chd[2]=curNd->chd[2];
52             newNd->chd[3]=curNd->chd[3];
53         }
54         if (root->chd[0]->dep < root->chd[1]->dep)
55             root->chd[0]=merge(root->chd[0], newNd);
56         else root->chd[1]=merge(root->chd[1], newNd);
57         root->dep=max(root->chd[0]->dep, root->chd[1]->dep)+1;
58         return root;
59     }
60     vector<heap*> V;
61     void build() {
62         nullNd=new heap; nullNd->dep=0; nullNd->edge=new nd;
63         fill(nullNd->chd, nullNd->chd+4, nullNd);
64         while (not dfsQ.empty()) {
65             int u=dfsQ.front(); dfsQ.pop();
66             if (!nxt[u]) head[u]=nullNd;
67             else head[u]=head[nxt[u]->v];
68             V.clear();
69             for (auto&& e: g[u]) {
70                 int v=e->v;
71                 if (dst[v] == -1) continue;
72                 e->d+=dst[v]-dst[u];
73                 if (nxt[u] != e) {
74                     heap* p=new heap; fill(p->chd, p->chd+4, nullNd);
75                     p->dep=1; p->edge=e; V.push_back(p);
76                 }
77             }
78             if (V.empty()) continue;
79             make_heap(V.begin(), V.end(), cmp);
80             #define L(X) ((X<<1)+1)

```

```

79 #define R(X) ((X<<1)+2)
80 for(size_t i=0;i<V.size();i++){
81     if(L(i)<V.size()) V[i]->chd[2]=V[L(i)];
82     else V[i]->chd[2]=nullNd;
83     if(R(i)<V.size()) V[i]->chd[3]=V[R(i)];
84     else V[i]->chd[3]=nullNd;
85 }
86 head[u]=merge(head[u],V.front());
87 } }
88 vector<ll> ans;
89 void first_K(){
90     ans.clear(); priority_queue<node> Q;
91     if(dst[s]==-1) return;
92     ans.push_back(dst[s]);
93     if(head[s]!=nullNd)
94         Q.push(node(head[s],dst[s]+head[s]->edge->d));
95     for(int _=1;_<k and not Q.empty();_++){
96         node p=Q.top();Q.pop(); ans.push_back(p.d);
97         if(head[p.H->edge->v]!=nullNd){
98             q.H=head[p.H->edge->v]; q.d=p.d+q.H->edge->d;
99             Q.push(q);
100         }
101         for(int i=0;i<4;i++){
102             if(p.H->chd[i]!=nullNd){
103                 q.H=p.H->chd[i];
104                 q.d=p.d+p.H->edge->d+p.H->chd[i]->edge->d;
105                 Q.push(q);
106             }
107         }
108     }
109     void solve(){ // ans[i] stores the i-th shortest path
110         dijkstra(); build();
111         first_K(); // ans.size() might less than k
112     }
113 } solver;

```

6.9 SCC - Tarjan

7a063d

```

1 // Author: Ian, https://judge.yosupo.jp/submission/399291
2 // Tarjan SCC, 0-based
3 // 1. SCC scc(n); // n = #(vertices)
4 // 2. Fill in adj list G
5 // 3. scc.build()
6 // => scc[i]: each scc
7 // => sccn[x]: in which scc is x
8 struct SCC {
9     const int inf = 1e9+5;
10    int n, t, ncnt;
11    vector<vector<int>>> G, scc;
12    vector<bool> ins;
13    vector<int> sccn, dfn, low, s;
14    SCC(int n): n(n), t(0), ncnt(0), G(n),
15        ins(n, false), sccn(n), dfn(n, -1), low(n, inf) {}
16    void dfs(int x) {
17        if (~dfn[x]) return;
18        dfn[x] = low[x] = t++;
19        s.push_back(x), ins[x] = true;
20        for (auto i: G[x])
21            if (dfs(i), ins[i])
22                low[x] = min(low[x], low[i]);
23        if (dfn[x] == low[x]) {
24            scc.resize(scc.size()+1);
25            int p; do {
26                ins[p = s.back()] = false;
27                s.pop_back(); sccn[p] = ncnt;
28                scc.back().push_back(p);
29            } while (p != x); ncnt++;
30        }
31    }
32    void build() { for (int i=0; i<n; i++) dfs(i); }
33 };

```

6.10 2SAT

130351

```

1 // Author: Ian, CRYPTOGRAPHER
2 struct TwoSAT: SCC {
3     TwoSAT(int n): SCC(2*n) {}
4     // (var a == na) v (var b == nb)
5     void add_disjunction(int a, int na, int b, int nb) {
6         a = 2*a^na, b = 2*b^nb;
7         G[a^1].push_back(b);
8         G[b^1].push_back(a);
9     }
10    vector<int> solve() {
11        build(); vector<int> assignment(n/2, 0);
12        for(int i=0; i<n/2; i++){
13            if (sccn[2*i] == sccn[2*i^1]) return {};
14            assignment[i] = sccn[2*i] > sccn[2*i^1];
15        } return assignment;
16    };

```

7 Tree

7.1 Tree Isomorphism (Rooted Trees)

f3f029

```

1 // Author: Gino
2 // Checks if 2 rooted trees are isomorphic
3 // Time: O(V log V)
4 // vector<vector<int>> G1, G2;
5 // int root_1, root_2;
6 map<vector<int>, int> dict;
7 int encode(const auto& G, int u, int pa = -1) {
8     vector<int> codes;
9     for (auto& v : G[u])
10         if (v != pa)
11             codes.emplace_back(encode(G, v, u));
12     ranges::sort(codes);
13     auto [it, _] = dict.try_emplace(codes, dict.size());
14     return it->second;
15 }
16 bool isomorphic() {
17     dict.clear(); dict[{}] = 0;
18     return encode(G1, root_1) == encode(G2, root_2);
19 }

```

7.2 Tree Isomorphism (Unrooted Trees)

Find the centroid(s) of T_1, T_2 .Case 1: T_1, T_2 have different number of centroids $\rightarrow$ NOCase 2: T_1 has centroid c_1, T_2 has centroid c_2 $\rightarrow$ rooted_isomorphic(c_1, c_2)Case 3: T_1 has centroids c_1, c'_1, T_2 has centroids c_2, c'_2 $\rightarrow$ rooted_isomorphic(c_1, c_2) || rooted_isomorphic(c'_1, c'_2)

7.3 Heavy Light Decomposition

53e45c

```

1 // Author: Ian, https://judge.yosupo.jp/submission/399317
2 void segbuild(vector<int>&v);
3 void segmodify(int p, int k);
4 int segquery(int ql, int qr);
5 // Insert [ql, qr] segment tree here
6 template<class T=int> struct HLD {
7     int n, cnt;
8     vector<int> d, f, sz, son, top, dfn;
9     HLD(int n): n(n), cnt(0), d(n, 0), f(n, 0), sz(n, 1),
10         son(n, -1), top(n, 0), dfn(n, -1) {}
11     void dfs1(int x, int p) {
12         for (auto i: e[x]) if (i != p) {
13             d[i] = d[x]+1, f[i] = x;
14             dfs1(i, x), sz[x] += sz[i];
15             if (son[x] == -1 || sz[son[x]] < sz[i])
16                 son[x] = i;
17         }
18     }
19     void dfs2(int x, int t) {
20         dfn[x] = cnt++, top[x] = t;
21         if (son[x] == -1) return;
22         dfs2(son[x], t);
23         for (auto i: e[x])
24             if (!~dfn[i]) dfs2(i, i);
25     }
26     void build(const vector<T> &v) {
27         dfs1(0, 0); dfs2(0, 0);
28         vector<T> dfnv(v.size());
29         for (int i=0; i<n; i++)
30             dfnv[dfn[i]] = v[i];
31         segbuild(dfnv);
32     }
33     void modify(int p, T v) { segmodify(dfn[p], v); }
34     T query(int a, int b) {
35         auto aggr = [](int x, int y) { return max(x, y); };
36         T ret = 0;
37         while (top[a] != top[b]) {
38             if (d[top[a]] > d[top[b]]) swap(a, b);
39             ret = aggr(ret, segquery(dfn[top[b]], dfn[b]+1));
40             b = f[top[b]];
41         }
42         if (dfn[a] > dfn[b]) swap(a, b);
43         return aggr(ret, segquery(dfn[a], dfn[b]+1));
44     }
45 };

```

7.4 Virtual Tree

172166

```

1 // Author: std_abs
2 // need lca, in, out
3 vector<pii> virtual_tree(vector<int> &v) {
4     auto cmp = [&](int x, int y) { return in[x] < in[y]; };
5     sort(all(v), cmp);
6     for (int i = 1; i < sz(v); ++i)
7         v.pb(lca(v[i-1], v[i]));
8     sort(all(v), cmp);
9     v.resize(unique(all(v)) - v.begin());
10    vector<int> stk(1, v[0]);
11    vector<pii> res;
12    for (int i = 1; i < sz(v); ++i) {
13        int x = v[i];

```

```

14 while (out[stk.back()] < out[x]) stk.pop_back();
15 res.emplace_back(stk.back(), x), stk.pb(x);
16 }
17 return res;
18 }

```

8 Matching

8.1 Bipartite Matching

f42921

```

1 // Author: Gino
2 // Function: Max Bipartite Matching in  $O(V \sqrt{E})$ 
3 //> hk.init(nx, ny) -> hk.add(x, y (+nx))
4 //> hk.max_matching(): the matching plan stores in mx, my
5 //> hk.min_vertex_cover(): vertex cover plan stores in vcover
6 // (!) vertices are 0-based: X = [0, nx), Y = [nx, nx+ny)
7 #define pb emplace_back
8 struct HopcroftKarp {
9     int n, nx, ny; vector<vector<int>> G; vector<int> mx, my;
10     void init(int nx, int ny): nx(nx), ny(ny), n(nx + ny) {
11         G.assign(n, vector<int>()); }
12     void add(int x, int y) { G[x].pb(y); G[y].pb(x); }
13     int max_matching() {
14         vector<int> dis, vis;
15         mx.assign(n, -1); my.assign(n, -1);
16         function<bool(int)> dfs = [&](int x) {
17             vis[x] = 1;
18             for (int y : G[x]) {
19                 int p = my[y];
20                 if (p == -1 ||
21                     (dis[p] == dis[x] + 1 && !vis[p] && dfs(p)))
22                     return mx[x] = y, my[y] = x, true;
23             }
24             return false;
25         };
26         while (true) {
27             dis.assign(n, -1);
28             queue<int> q;
29             for (int x = 0; x < nx; x++)
30                 if (mx[x] == -1) dis[x] = 0, q.push(x);
31             while (!q.empty()) {
32                 int x = q.front(); q.pop();
33                 for (int y : G[x])
34                     if (my[y] != -1 && dis[my[y]] == -1)
35                         dis[my[y]] = dis[x] + 1, q.push(my[y]);
36             }
37             vis.assign(n, 0);
38             bool found = false;
39             for (int x = 0; x < nx; x++)
40                 if (mx[x] == -1 && dfs(x)) found = true;
41             if (!found) break;
42         }
43         int ans = 0;
44         for (int x = 0; x < nx; x++) if (mx[x] != -1) ans++;
45         return ans;
46     }
47     vector<int> vcover;
48     int min_vertex_cover() {
49         int ans = max_matching(); vector<int> vis(n, 0);
50         function<void(int)> dfs = [&](int x) {
51             vis[x] = true;
52             for (int y : G[x]) {
53                 if (y == mx[x] || my[y] == -1 || vis[y]) continue;
54                 vis[y] = true;
55                 dfs(my[y]);
56             }
57         };
58         for (int x = 0; x < nx; x++) if (mx[x] == -1) dfs(x);
59         vcover.clear();
60         for (int x = 0; x < nx; x++) if (!vis[x]) vcover.pb(x);
61         for (int y = nx; y < nx+ny; y++) if (vis[y]) vcover.pb(y);
62         return ans;
63     };
64 }; hk;

```

8.2 Bipartite Weighted Matching

5933aa

```

1 // Author: ckiseki, n = #(left vertices) = #(right vertices)
2 // left, right vertices labeled 1 ~ n separately
3 // 1. build 1-based matrix G of size (n + 1) * (n + 1)
4 // G[u][v] := weight between (left u, right v)
5 // <!-- if not need perfect matching
6 // ==> set weights of all negative edges to 0
7 // 2. construct (solve at same time): KM km(n, G);
8 // ==> km.ans (max perfect weight sum)
9 // ==> km.fl[u] (left u -> right v)
10 // ==> km.fr[v] (right v -> left u)
11 // Time:  $O(V^3)$ , Space:  $O(V^2)$ , Test:  $V \leq 500$ , 243ms
12 struct KM { // maximize, test @ UOJ 80
13     int n, l, r; ll ans; // fl and fr are the match
14     vector<ll> hl, hr; vector<int> fl, fr, pre, q;
15     void bfs(const auto &w, int s) {
16         vector<int> vl(n + 1), vr(n + 1);

```

```

17     vector<ll> slk(n + 1, LINF);
18     l = r = 0; vr[q[r++] = s] = true;
19     auto check = [&](int x) -> bool {
20         if (vl[x] || slk[x] > 0) return true;
21         vl[x] = true; slk[x] = LINF;
22         if (fl[x] != -1) return (vr[q[r++] = fl[x]] = true);
23         while (x != -1) swap(x, fr[fl[x] = pre[x]]);
24         return false;
25     };
26     while (true) {
27         while (l < r)
28             for (int x = 1, y = q[l++]; x <= n; ++x) if (!vl[x])
29                 if (ll val = hl[x] + hr[y] - w[x][y], val < slk[x]) {
30                     slk[x] = val;
31                     if (pre[x] = y, !check(x)) return;
32                 }
33         ll d = LINF;
34         for (int x = 1; x <= n; ++x) d = min(d, slk[x]);
35         for (int x = 1; x <= n; ++x)
36             vl[x] ? hl[x] += d : slk[x] -= d;
37         for (int x = 1; x <= n; ++x) if (vr[x]) hr[x] -= d;
38         for (int x = 1; x <= n; ++x) if (!check(x)) return;
39     }
40     KM(int n_, const auto &w) : n(n_), ans(0),
41         hl(n+1, hr(n+1), fl(n+1, -1), fr(fl), pre(n+1), q(n+1) {
42         for (int i = 1; i <= n; ++i) {
43             hl[i] = -LINF;
44             for (int j = 1; j <= n; ++j) hl[i] = max(hl[i], (ll)w[i][j]);
45         }
46         for (int i = 1; i <= n; ++i) bfs(w, i);
47         for (int i = 1; i <= n; ++i) ans += w[i][fl[i]];
48     }
49 };

```

8.3 General Matching

93f948

```

1 // Author: ckiseki
2 // 0. build 1-based graph vector<vector<int>> G(n + 1);
3 // 1. construct (solve at same time):
4 // GeneralMatching M(G, n); ==> M.ans
5 // status: M.match[x] (== -1 if not matched, 1 <= x <= n)
6 // Time:  $O(V^3)$ , Space:  $O(V + E)$ , Test:  $V \leq 500$ , 11ms
7 struct GeneralMatching {
8     queue<int> q; int ans, n;
9     vector<int> fa, s, v, pre, match;
10     int Find(int u) {
11         return u == fa[u] ? u : fa[u] = Find(fa[u]); }
12     int LCA(int x, int y) {
13         static int tk = 0; tk++; x = Find(x); y = Find(y);
14         for (; swap(x, y)) if (x != 0) {
15             if (v[x] == tk) return x;
16             v[x] = tk;
17             x = Find(pre[match[x]]);
18         }
19     }
20     void Blossom(int x, int y, int l) {
21         for (; Find(x) != l; x = pre[y]) {
22             pre[x] = y, y = match[x];
23             if (s[y] == 1) q.push(y), s[y] = 0;
24             for (int z : {x, y}) if (fa[z] == z) fa[z] = l;
25         }
26     }
27     bool Bfs(auto &&g, int r) {
28         iota(all(fa), 0); ranges::fill(s, -1);
29         q = queue<int>(); q.push(r); s[r] = 0;
30         for (; !q.empty(); q.pop()) {
31             for (int x = q.front(); int u : g[x])
32                 if (s[u] == -1) {
33                     if (pre[u] = x, s[u] = 1, match[u] == 0) {
34                         for (int a = u, b = x, last;
35                             b != 0; a = last, b = pre[a])
36                             last = match[b], match[b] = a, match[a] = b;
37                         return true;
38                     }
39                     q.push(match[u]); s[match[u]] = 0;
40                 }
41             else if (!s[u] && Find(u) != Find(x)) {
42                 int l = LCA(u, x);
43                 Blossom(x, u, l); Blossom(u, x, l);
44             }
45         }
46         return false;
47     }
48     GeneralMatching(auto &&g, int n_) : ans(0), n(n_),
49         fa(n+1), s(n+1), v(n+1), pre(n+1, 0), match(n+1, 0) {
50         for (int x = 1; x <= n; ++x)
51             if (match[x] == 0) ans += Bfs(g, x);
52         for (int x = 1; x <= n; ++x)
53             if (match[x] == 0) match[x] = -1;
54     }
55 };

```

9 Flow

9.1 Flow Methods

c5090e

```

1 // Author: CRYPTOGRAPHER (some modified by Gino)
2 Maximize  $c^T x$  subject to  $Ax \leq b, x \geq 0$ ;
3 with the corresponding symmetric dual problem,
4 Minimize  $b^T y$  subject to  $A^T y \geq c, y \geq 0$ .
5
6 Maximize  $c^T x$  subject to  $Ax \leq b$ ;
7 with the corresponding asymmetric dual problem,
8 Minimize  $b^T y$  subject to  $A^T y = c, y \geq 0$ .
9
10 Maximize  $\sum x$  subject to  $x_i + x_j \leq A_{ij}, x \geq 0$ ;
11  $\Rightarrow$  Maximize  $\sum x$  subject to  $x_i + x_j \leq A_{ij}$ ;
12  $\Rightarrow$  Minimize  $A^T y = \sum A_{ij} y_{ij}$  subject to for all  $v$ ,
13  $\sum_{i=v \text{ or } j=v} y_{ij} = 1, y_{ij} \geq 0$ 
14  $\Rightarrow$  possible optimal solution:  $y_{ij} = \{0, 0.5, 1\}$ 
15  $\Rightarrow y' = 2y$ :  $\sum_{i=v \text{ or } j=v} y'_{ij} = 2, y'_{ij} = \{0, 1, 2\}$ 
16  $\Rightarrow$  Minimum Bipartite perfect matching/2 ( $V_1=X, V_2=X, E=A$ )
17
18 General Graph:
19 |Max Ind. Set| + |Min Vertex Cover| = |V|
20 |Max Ind. Edge Set| + |Min Edge Cover| = |V|
21 Bipartite Graph:
22 |Max Ind. Set| = |Min Edge Cover|
23 |Max Ind. Edge Set| = |Min Vertex Cover|
24
25 To reconstruct the minimum vertex cover, dfs from each
26 unmatched vertex on the left side and with unused edges
27 only. Equivalently, dfs from source with unused edges
28 only and without visiting sink. Then, a vertex is
29 chosen iff. it is on the left side and without visited
30 or on the right side and visited through dfs.
31
32 Minimum Weighted Bipartite Edge Cover:
33 Construct new bipartite graph with  $n+m$  vertices on each
34 side:
35 for each vertex  $u$ , duplicate a vertex  $u'$  on the other side
36 for each edge  $(u,v,w)$ , add edges  $(u,v,w)$  and  $(v',u',w)$ 
37 for each vertex  $u$ , add edge  $(u,u',2w)$  where  $w$  is min edge
38 connects to  $u$ 
39 then the answer is the minimum perfect matching of the new
40 graph (KM)
41
42 Maximum density subgraph (  $\sum W_e + \sum W_v$  ) / |V|
43 Binary search on answer:
44 For a fixed  $D$ , construct a Max flow model as follow:
45 Let  $S$  be Sum of all weight( or inf)
46 1. from source to each node with cap =  $S$ 
47 2. For each  $(u,v,w)$  in  $E$ ,  $(u \rightarrow v, \text{cap}=w), (v \rightarrow u, \text{cap}=w)$ 
48 3. For each node  $v$ , from  $v$  to sink with cap =  $S + 2 * D - \text{deg}[v] - 2 * (W \text{ of } v)$ 
49 where  $\text{deg}[v] = \sum \text{weight of edge associated with } v$ 
50 If  $\text{maxflow} < S * |V|$ ,  $D$  is an answer.
51
52 Requiring subgraph: all vertex can be reached from source
53 with
54 edge whose cap  $> 0$ .
55
56 Maximum closed subgraph
57 1. connect source with positive weighted
58 vertex(capacity=weight)
59 2. connect sink with negative weighted vertex(capacity=-
60 weight)
61 3. make capacity of the original edges = inf
62 4. ans = sum(positive weighted vertex weight) - (max flow)
63
64 (Node-disjoint) Min DAG Path Cover (用最少路徑覆蓋所有點)
65 Node disjoint: 拆出來的路徑不能共用同一個點
66 將一個點  $u$  裂成  $u_+$  和  $u_-$ , 代表進入和出去
67 對於一條邊  $(u, v)$  建邊  $u_- \Rightarrow v_+$ 
68 1.  $+$ 點們和  $-$ 點們形成一張二分圖
69 2. 最差的答案是  $n$ , 代表每個點自己一個點就是一條路徑
70 3. 只要一組  $(u_-, v_+)$  匹配成功那對應到的答案恰好會  $-1$ 
71  $\ggg$  ans =  $n -$  這張二分圖的最大匹配
72
73 General DAG Path Cover
74 跟 Node-disjoint 版本差在點可以被很多條路共用
75 建邊方式 Node-disjoint 差別在條件比較鬆
76 Node-disjoint:  $u_- \Rightarrow v_+$  只能在  $(u, v)$  有邊時建
77 General:  $u_- \Rightarrow v_+$  在  $u$  能走到  $v$  的時候就建
78
79 Dilworth Theorem
80 反鏈：一些節點的集合，滿足這些節點互相無法抵達
81 最大反鏈 = 最小 General DAG Path Cover
82
83 Min Cut reconstruction (after running max flow  $s \rightarrow t$ ):
84 From  $s$ , BFS/DFS through residual edges with remaining
85 capacity  $> 0$ , marking reached vertices as visited.
86 The cut set is every original edge  $(u, v)$  with  $u$  visited
87 and  $v$  not visited; its capacity sum equals the max flow.

```

```

81
82 Bipartite Matching reconstruction (after running max flow
83 on source  $\rightarrow X \rightarrow Y \rightarrow$  sink model):
84 For each edge  $(x, y)$  between the  $X$ -side and  $Y$ -side, it is
85 part of the matching iff. its residual capacity is 0
86 (i.e. it is saturated).
87
88 Max/Min Flow with Lower Bound (Circulation Problem):
89 1. Construct super source  $S$  and super sink  $T$ .
90 2. For each edge  $(x, y, l, u)$ , connect  $x \rightarrow y$  with capacity
91  $u - l$ .
92 3. For each vertex  $v$ , let  $\text{in}(v) = (\text{sum of incoming lower}$ 
93  $\text{bounds})$ 
94  $- (\text{sum of outgoing lower bounds})$ .
95 4. If  $\text{in}(v) > 0$ , connect  $S \rightarrow v$  with capacity  $\text{in}(v)$ ;
96 otherwise connect  $v \rightarrow T$  with capacity  $-\text{in}(v)$ .
97 To maximize: connect  $t \rightarrow s$  with capacity inf (skip this
98 edge
99 for a pure circulation problem), let  $f = \text{maxflow}(S, T)$ .
100 If
101  $f \neq \text{sum of in}(v) \text{ over all } \text{in}(v) > 0$ , no solution exists.
102 Otherwise  $\text{maxflow}(s, t)$  (found after adding  $t \rightarrow s$ ) is
103 the answer.
104 To minimize: let  $f = \text{maxflow}(S, T)$  (without any  $s-t$  edge
105 yet).
106 Then connect  $t \rightarrow s$  with capacity inf and let  $f'$  be the
107 extra
108 flow pushed from  $S$  to  $T$ . If  $f + f' \neq \text{sum of in}(v) \text{ over}$ 
109  $\text{all}$ 
110  $\text{in}(v) > 0$ , no solution exists. Otherwise  $f'$  is the
111 answer.
112 5. The flow on each original edge  $e$  is  $l_e + f_e$ , where  $f_e$ 
113 is
114  $e$ 's flow in the constructed graph.

```

9.2 Dinic (Flow Decomposition)

b8e775

```

1 // Function: Max Flow + Flow Decomposition,  $O(V^2 E)$ 
2 // init  $\rightarrow$  addEdge; vertices 0-based
3 //  $\ggg$  flow() := return max flow
4 //  $\ggg$  flow_decomposition() := return max flow + store
5 // decomposition in flow.D
6
7 struct Dinic{
8     static const int MXN=10000;
9     struct Edge{ int v,c,r,C; bool fw; };
10    int n,s,t,level[MXN]; vector<Edge> G[MXN];
11    void init(int _n,int _s,int _t){
12        n=_n; s=_s; t=_t;
13        for (int i=0; i<n; i++) G[i].clear();
14    }
15    void addEdge(int u,int v,int c){
16        G[u].push_back({v,c,(int)G[v].size(),c,true});
17        G[v].push_back({u,0,(int)G[u].size()-1,0,false});
18    }
19    bool BFS(){
20        for (int i=0; i<n; i++) level[i]=-1;
21        queue<int> que; que.push(s); level[s]=0;
22        while(!que.empty()){
23            int u=que.front(); que.pop();
24            for (auto &it:G[u]){
25                if (it.c>0&&level[it.v]==-1){
26                    level[it.v]=level[u]+1; que.push(it.v);
27                }
28            }
29            return level[t]!=-1;
30        }
31    }
32    int DFS(int u,int nf){
33        if(u==t) return nf;
34        int res=0;
35        for (auto &it:G[u]){
36            if (it.c>0&&level[it.v]==level[u]+1){
37                int tf=DFS(it.v,min(nf,it.c));
38                res+=tf; nf-=tf; it.c-=tf;
39                G[it.v][it.r].c+=tf;
40                if(nf==0) return res;
41            }
42        }
43        if(!res) level[u]=-1;
44        return res;
45    }
46    int flow(int res=0){
47        while(BFS()) res+=DFS(s,2147483647);
48        return res;
49    }
50    // reset: do nothing
51    vector<pair<int, vector<int>>> D; // (flow amount, [path
52    p1 ... pk])
53    int flow_decomposition(){
54        int mxflow = flow();
55        vector<vector<Edge>> fG(n); // graph with forward edges
56        for (int u = 0; u < n; u++) {
57            for (auto &e : G[u]) {
58                if (e.fw && e.C - e.c > 0)
59                    fG[u].push_back({e.v, e.C - e.c, e.r, e.C, e.fw});
60            }
61        }
62        vector<int> vis(n, 0);
63        function<int(int, int)> dfs = [&](int u, int cur) {
64            if (u == t) {

```

```

53     D.back().second.push_back(u);
54     return cur; }
55 vis[u] = 1;
56 for (auto &e : fg[u]) {
57     if (e.c > 0 && !vis[e.v]) {
58         int ans = dfs(e.v, min(cur, e.c));
59         if (ans > 0) {
60             e.c -= ans;
61             D.back().second.push_back(u);
62             return ans;
63         } } }
64 return 0; };
65 D.clear();
66 int quota = mxflow;
67 while (quota > 0) {
68     D.emplace_back(0, vector<int>());
69     fill(vis.begin(), vis.end(), 0);
70     int f = dfs(s, 2147483647);
71     if (f == 0) break;
72     reverse(D.back().second.begin(),
73            D.back().second.end());
74     D.back().first = f, quota -= f;
75 }
76 return mxflow; }
77 }flow;

```

9.3 ISAP

c13d55

```

1 // Author: CRYPTOGRAPHER
2 // 1-based: vertices are numbered 1 ~ n
3 // 1. flow.init(n, s, t);
4 // n = #vertices, s, t = source, sink
5 // 2. flow.addEdge(u, v, c); // u, v in [1, n]
6 // 3. call: int ans = flow.flow();
7 // => ans = max flow value from s to t
8 // Time: O(V^2 * E) worst case, usually much faster in
9 // practice
10 static const int MAXV=50010;
11 static const int INF =1000000;
12 struct Maxflow{
13     struct Edge{
14         int v,c,r;
15         Edge(int _v,int _c,int _r):v(_v),c(_c),r(_r){}
16     };
17     int s,t; vector<Edge> G[MAXV];
18     int iter[MAXV],d[MAXV],gap[MAXV],tot;
19     void init(int n,int _s,int _t){
20         tot=n,s=_s,t=_t;
21         for(int i=1;i<=tot;i++){
22             G[i].clear(); iter[i]=d[i]=gap[i]=0;
23         }
24     }
25     void addEdge(int u,int v,int c){
26         G[u].push_back(Edge(v,c,SZ(G[v])));
27         G[v].push_back(Edge(u,0,SZ(G[u])-1));
28     }
29     int DFS(int p,int flow){
30         if(p==t) return flow;
31         for(int &i=iter[p];i<SZ(G[p]);i++){
32             Edge &e=G[p][i];
33             if(e.c>0&&d[p]==d[e.v]+1){
34                 int f=DFS(e.v,min(flow,e.c));
35                 if(f){ e.c-=f; G[e.v][e.r].c+=f; return f; }
36             }
37         }
38         if(--gap[d[p]]==0) d[s]=tot;
39         else{ d[p]++; iter[p]=0; ++gap[d[p]]; }
40         return 0;
41     }
42     int flow(){
43         int res=0;
44         for(res=0,gap[0]=tot;d[s]<tot;res+=DFS(s,INF));
45         return res;
46     }
47     void reset(){
48         for(int i=1;i<=tot;i++) iter[i]=d[i]=gap[i]=0;
49     }
50 }flow;

```

9.4 Bounded Max Flow

aaf8c3

```

1 // Author: CRYPTOGRAPHER
2 // Max flow with lower/upper bound on edges (source-sink
3 // version)
4 // 1-based: original vertices are numbered 1 ~ n.
5 // Super source, sink: n+1, n+2 respectively.
6 // dependency: ISAP.cpp
7 // 1. n = #vertices (1 ~ n), m = #edges
8 // s, t = original source, sink (both in [1, n])
9 // 2. l[i], r[i] := edge i goes from l[i] -> r[i] (l[i],
10 // r[i] in [1, n])
11 // 3. a[i], b[i] := flow on edge i must lie in [a[i], b[i]]

```

```

10 // 4. int ans = solve(n, m, s, t);
11 // => ans == -1 : no feasible flow exists
12 // => ans : feasible max flow value
13 // <!=> N, M must be large enough for original graph + 2
14 // super nodes
15 // Time: O(V^2 E), Space: O(V + E)
16 int in[N], out[N], l[M], r[M], a[M], b[M];
17 int solve(int n, int m, int s, int t){
18     flow.init(n+2, n+1, n+2); // super source = n+1, super
19     sink = n+2
20     for(int i = 0; i < m; i++){
21         in[r[i]] += a[i]; out[l[i]] += a[i];
22         flow.addEdge(l[i], r[i], b[i] - a[i]);
23         // flow from l[i] to r[i] must lie in [a[i], b[i]]
24     }
25     int nd = 0;
26     for(int i = 1; i <= n; i++){
27         if(in[i] < out[i]){
28             flow.addEdge(i, flow.t, out[i] - in[i]);
29             nd += out[i] - in[i];
30         }
31         if(out[i] < in[i])
32             flow.addEdge(flow.s, i, in[i] - out[i]);
33     }
34     flow.addEdge(t, s, INF); // original sink -> source
35     if(flow.flow() != nd) return -1; // infeasible
36     int ans = flow.G[s].back().c; // t->s edge's current flow
37     = feasible flow s->t
38     flow.G[s].back().c = flow.G[t].back().c = 0;
39     for(size_t i = 0; i < flow.G[flow.s].size(); i++){
40         Maxflow::Edge &e = flow.G[flow.s][i];
41         flow.G[flow.s][i].c = 0; flow.G[e.v][e.r].c = 0;
42     }
43     for(size_t i = 0; i < flow.G[flow.t].size(); i++){
44         Maxflow::Edge &e = flow.G[flow.t][i];
45         flow.G[flow.t][i].c = 0; flow.G[e.v][e.r].c = 0;
46     }
47     flow.addEdge(flow.s, s, INF); flow.addEdge(t, flow.t,
48     INF);
49     flow.reset(); // keeps G[] intact, only resets search
50     state
51     return ans + flow.flow();
52 }

```

9.5 MCMF

472be6

```

1 // Author: CRYPTOGRAPHER
2 // MCMF (supports negative edges without negative cycles)
3 // <!=> 1-based: vertices are numbered 1 ~ n.
4 // 1. MCMF.init(n); // n = #vertices (1 ~ n)
5 // 2. MCMF.add(u, v, cap, cost);
6 // 3. auto [max_flow, min_cost] = MCMF.flow(s, t); // s, t
7 // in [1, n]
8 // Time: O(F * E), Space: O(V + E)
9 typedef int Tcost;
10 const int MAXV = 20010;
11 const int INFf = 1000000;
12 const Tcost INFc = 1e9;
13 struct MCMF {
14     struct Edge{
15         int v, cap;
16         Tcost w;
17         int rev;
18         bool fw;
19         Edge(){}
20         Edge(int t2, int t3, Tcost t4, int t5, bool t6)
21             : v(t2), cap(t3), w(t4), rev(t5), fw(t6) {}
22     };
23     int V, s, t;
24     vector<Edge> G[MAXV];
25     void init(int n){
26         V = n;
27         for(int i = 0; i <= V; i++) G[i].clear();
28     }
29     void add(int a, int b, int cap, Tcost w){
30         G[a].push_back(Edge(b, cap, w, (int)G[b].size(), true));
31         G[b].push_back(Edge(a, 0, -w, (int)G[a].size()-1,
32         false));
33     }
34     Tcost d[MAXV];
35     int id[MAXV], mom[MAXV];
36     bool inqu[MAXV];
37     queue<int> q;
38     pair<int, Tcost> flow(int _s, int _t){
39         s = _s, t = _t;
40         int mxflow = 0, Tcost mnc = 0;
41         while(1){
42             fill(d, d+V, INFc); // need to use type cast
43             fill(inqu, inqu+V, 0);
44             fill(mom, mom+V, -1);
45             mom[s] = s;

```

```

44 d[s] = 0;
45 q.push(s); inq[u[s] = 1;
46 while(q.size()){
47     int u = q.front(); q.pop();
48     inq[u] = 0;
49     for(int i = 0; i < (int) G[u].size(); i++){
50         Edge &e = G[u][i];
51         int v = e.v;
52         if(e.cap > 0 && d[v] > d[u]+e.w){
53             d[v] = d[u]+e.w;
54             mom[v] = u;
55             id[v] = i;
56             if(!inq[v]) q.push(v), inq[v] = 1;
57         }
58     }
59 }
60 if(mom[t] == -1) break;
61 int df = INFF;
62 for(int u = t; u != s; u = mom[u])
63     df = min(df, G[mom[u]][id[u]].cap);
64 for(int u = t; u != s; u = mom[u]){
65     Edge &e = G[mom[u]][id[u]];
66     e.cap -= df;
67     G[e.v][e.rev].cap += df;
68 }
69 mxf += df;
70 mnc += df*d[t];
71 }
72 return make_pair(mxf, mnc);
73 }
74 };

```

9.6 Push-Relabel

fae237

```

1 // Author: Simon Lindholm (kactl)
2 // 0-based: vertices are numbered 0 ~ n-1
3 // 1. PushRelabel pr(n); // n = #vertices
4 // 2. pr.addEdge(u, v, c); // u, v in [0, n-1]
5 // 3. ll ans = pr.calc(s, t);
6 // => ans = max flow value from s to t
7 // To obtain actual flow:
8 // for (int u = 0; u < n; u++)
9 //     for (auto& e : pr.g[u])
10 //         if (e.f > 0) => (u, e.dest, e.f)
11 // Time:  $O(V^2 * \sqrt{E})$ , better on dense graph
12 struct PushRelabel {
13     struct Edge { int dest, back; ll f, c; };
14     vector<vector<Edge>> g;
15     vector<ll> ec;
16     vector<Edge*> cur;
17     vector<vi> hs; vi H;
18     PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}
19
20     void addEdge(int s, int t, ll cap, ll rcap=0) {
21         if (s == t) return;
22         g[s].push_back({t, sz(g[t]), 0, cap});
23         g[t].push_back({s, sz(g[s])-1, 0, rcap});
24     }
25
26     void addFlow(Edge& e, ll f) {
27         Edge &back = g[e.dest][e.back];
28         if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
29         e.f += f; e.c -= f; ec[e.dest] += f;
30         back.f -= f; back.c += f; ec[back.dest] -= f;
31     }
32
33     ll calc(int s, int t) {
34         int v = sz(g); H[s] = v; ec[t] = 1;
35         vi co(2*v); co[0] = v-1;
36         rep(i, 0, v) cur[i] = g[i].data();
37         for (Edge& e : g[s]) addFlow(e, e.c);
38
39         for (int hi = 0; ) {
40             while (hs[hi].empty()) if (!hi--) return -ec[s];
41             int u = hs[hi].back(); hs[hi].pop_back();
42             while (ec[u] > 0) // discharge u
43                 if (cur[u] == g[u].data() + sz(g[u])) {
44                     H[u] = 1e9;
45                     for (Edge& e : g[u]) if (e.c && H[u] >
46                         H[e.dest]+1)
47                         H[u] = H[e.dest]+1, cur[u] = &e;
48                     if (++co[H[u]], !--co[hi] && hi < v)
49                         rep(i, 0, v) if (hi < H[i] && H[i] < v)
50                             --co[H[i]], H[i] = v + 1;
51                     hi = H[u];
52                 } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
53                     addFlow(*cur[u], min(ec[u], cur[u]->c));
54                 else ++cur[u];
55             }
56         }
57         bool leftOfMinCut(int a) { return H[a] >= sz(g); }
58 };

```

9.7 Gomory-Hu Tree

88b014

```

1 // Author: chilli, Takanori MAEHARA (kactl)
2 // <!> Dependency: PushRelabel.cpp
3 // Given a list of edges representing an undirected flow
4 // graph,
5 // returns edges of the Gomory-Hu tree.
6 // maxflow / mincut of (u, v) => min edge from u to v on
7 // Gomory-Hu tree
8 // Time:  $O(V) * O(\text{PushRelabel})$ 
9 typedef array<ll, 3> Edge; // (u, v, w)
10 vector<Edge> gomoryHu(int N, vector<Edge> ed) {
11     vector<Edge> tree;
12     vi par(N);
13     rep(i, 1, N) {
14         PushRelabel D(N); // Dinic also works
15         for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
16         tree.push_back({i, par[i], D.calc(i, par[i])});
17         rep(j, i+1, N)
18             if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
19     }
20     return tree;
21 }

```

9.8 Global Min Cut

a040ee

```

1 // Author: ckiseki
2 // 1-based: vertices are numbered 1 ~ n
3 // 1. vector<vector<ll>> G(n+1, vector<ll>(n+1, 0));
4 // 2. add_edge(G, u, v, c); // add undirected edge u-v with
5 // weight c, u,v in [1,n]
6 // 3. ll ans = mincut(G, n); // n = #vertices
7 // => ans = global min cut value
8 // Time:  $O(V^3)$ 
9 void add_edge(auto &w, int u, int v, int c) {
10     w[u][v] += c; w[v][u] += c; }
11 auto phase(const auto &w, int n, vector<int> id) {
12     vector<ll> g(n+1); int s = -1, t = -1;
13     while (!id.empty()) {
14         int c = -1;
15         for (int i : id) if (c == -1 || g[i] > g[c]) c = i;
16         s = t; t = c;
17         id.erase(ranges::find(id, c));
18         for (int i : id) g[i] += w[c][i];
19     }
20     return tuple{s, t, g[t]};
21 }
22 ll mincut(auto w, int n) {
23     ll cut = numeric_limits<ll>::max();
24     vector<int> id(n); iota(all(id), 1);
25     for (int i = 0; i < n-1; ++i) {
26         auto [s, t, gt] = phase(w, n, id);
27         id.erase(ranges::find(id, t));
28         cut = min(cut, gt);
29         for (int j = 1; j <= n; ++j)
30             w[s][j] += w[t][j], w[j][s] += w[j][t];
31     }
32     return cut;
33 }

```

10 String

10.1 Rolling Hash

c15659

```

1 // Author: ckiseki (Modified by Gino)
2 // Usage: RH<Sigma, Mod> Hs(s);
3 // If no modifications needed, replace BIT with prefix sum
4 template <ll P = 127, ll Q = 1051762951>
5 class RH {
6     ll n; vector<ll> p, inv, d, bit;
7     int add(ll a, ll b) { return a+b >= Q ? a+b-Q : a+b; }
8     ll sub(ll a, ll b) { return a-b < 0 ? a-b+Q : a-b; }
9     ll mul(ll a, ll b) { return (__int128)a * b % Q; }
10     ll pw(ll a, ll x) {
11         ll r = 1; for (; x; a = mul(a, a), x >>= 1)
12             if (x & 1) r = mul(r, a);
13         return r;
14     }
15     void bit_add(int i, ll x) {
16         for (; i <= n; i += i&1) bit[i] = add(bit[i], x); }
17     ll bit_qry(int i) {
18         ll s = 0; for (; i; i -= i&1) s = add(s, bit[i]);
19     }
20     public:
21     RH(const auto &s) : n(s.size()),
22         p(n+1, 1), inv(n+1, 1), d(n+1), bit(n+1) {
23         ll iv = pw(P, Q-2);
24         for (int i = 1; i <= n; i++) {
25             p[i] = mul(p[i-1], P); inv[i] = mul(inv[i-1], iv);
26             d[i] = mul(s[i-1], p[i]); bit_add(i, d[i]);
27         }
28         void update(int i, char c) {
29             ll x = mul(c, p[i]); bit_add(i, sub(x, d[i])); d[i] = x; }
30         ll query(int l, int r) {
31             return mul(sub(bit_qry(r), bit_qry(l-1)), inv[l]); }
32     };

```

31};

10.2 KMP

376cdf

```
1// Author: Gino
2vector<int> kmp(auto s){ int n=s.size(); vector<int> f(n,0);
3  for (int i = 1, ptr = 0; i < n; i++) {
4    while (ptr && s[i] != s[ptr]) ptr = f[ptr - 1];
5    if (s[i] == s[ptr]) ptr++; f[i] = ptr;
6  } return f;
7}
```

10.3 Z Value

d5b3b9

```
1// Author: std_abs
2vector<int> Z(auto s){ int n=s.size(); vector<int> Z(n);
3  for (int i = 0, l = 0, r = 0; i < n; ++i) {
4    Z[i] = max(min(Z[i - l], r - i), 0);
5    while (i + Z[i] < n && s[Z[i]] == s[i + Z[i]])
6      l = i, r = i + Z[i], Z[i]++;
7  } return Z;
8}
```

10.4 Manacher

193b08

```
1// Author: std_abs
2// return array Z (length == 2N - 1)
3// Example: s = "aaaa"
4//   a       a       a
5// Z_0 Z_1 Z_2 Z_3 Z_4 Z_5 Z_6
6//   1   2   3   4   3   2   1
7// The corresponding palindrome s[l..r]:
8// l = (i - Z[i] + 1) / 2
9// r = (i + Z[i] - 1) / 2
10vector<int> manacher(auto tmp) {
11  vector<int> s(1, -1);
12  for (auto c : tmp) s.push_back(c), s.push_back(-1);
13  int l = 0, r = 0, n = (int)s.size();
14  vector<int> Z(n);
15  for (int i = 0; i < n; ++i) {
16    Z[i] = r > i ? min(Z[2 * l - i], r - i) : 1;
17    while (i - Z[i] >= 0 && i + Z[i] < n &&
18      s[i + Z[i]] == s[i - Z[i]]) Z[i]++;
19    if (Z[i] + i > r) l = i, r = Z[i] + i;
20  }
21  for (int i = 0; i < n; ++i)
22    Z[i] = (Z[i] - (i & 1)) / 2 * 2 + (i & 1);
23  return vector<int>(Z.begin() + 1, Z.end() - 1);
24}
```

10.5 Suffix Array + LCP

c1df5b

```
1// Author: std_abs
2// Tested @yosupo, N <= 5e5, 123ms
3const int N = 5e5 + 10;
4int sa[N], tmp[2][N], c[N], rk[N], lcp[N];
5void buildSA(string s) {
6  int *x = tmp[0], *y = tmp[1], m = 256, n = s.size();
7  for (int i = 0; i < m; ++i) c[i] = 0;
8  for (int i = 0; i < n; ++i) c[x[i]] = s[i]++;
9  for (int i = 1; i < m; ++i) c[i] += c[i - 1];
10 for (int i = n - 1; ~i; --i) sa[--c[x[i]]] = i;
11 for (int k = 1; k < n; k <= 1) {
12   for (int i = 0; i < m; ++i) c[i] = 0;
13   for (int i = 0; i < n; ++i) c[x[i]]++;
14   for (int i = 1; i < m; ++i) c[i] += c[i - 1];
15   int p = 0;
16   for (int i = n - k; i < n; ++i) y[p++] = i;
17   for (int i = 0; i < n; ++i) if (sa[i] >= k)
18     y[p++] = sa[i] - k;
19   for (int i = n - 1; ~i; --i)
20     sa[--c[x[y[i]]]] = y[i];
21   y[sa[0]] = p = 0;
22   for (int i = 1; i < n; ++i) {
23     int a = sa[i], b = sa[i - 1];
24     p += !(x[a] == x[b] && a + k < n && b + k < n && x[a + k] == x[b + k]);
25     y[sa[i]] = p;
26   }
27   if (n == p + 1) break;
28   swap(x, y), m = p + 1;
29 } }
30 void buildLCP(string s) {
31   // lcp[i] = LCP(sa[i - 1], sa[i])
32   // lcp(i, j) = query_lcp_min [rk[i] + 1, rk[j] + 1)
33   int n = s.length(), x = 0;
34   for (int i = 0; i < n; ++i) rk[sa[i]] = i;
35   for (int i = 0; i < n; ++i) {
36     if (!rk[i]) lcp[rk[i]] = 0;
37     else { if (x) x--;
38            int p = sa[rk[i] - 1];
39            while (x + i < n && x + p < n && s[x + i] == s[x + p]) x++;
40            lcp[rk[i]] = x;
41     } } }
```

10.6 Suffix Automaton

10f751

```
1// Author: std_abs
2// state (i.e. node) -> all substr with the same endpos
3// those substrs' length in range [len(link) + 1, len]
4// state i's endpos set -> union of pos[i] over the subtree
  of i in the suffix link tree
5// link[i] -> longest suffix with different endpos set
  (suffix link tree edge)
6// len[i] -> length longest suffix of state i
7// pos[i] -> an example end position of state i (1-based)
8// cnt[i] -> size of endpos set of state i, built by
  build_cnt() (tree DP)
9const int N = 5e5 + 10, C = 26;
10struct SAM {
11  int ch[2*N][C], len[2*N], link[2*N], pos[2*N], cnt[2*N], _id;
12  SAM () { reset(); }
13  void reset() { _id = 0, newnode(), link[0] = -1; }
14  int newnode() {
15    fill_n(ch[_id], C, 0);
16    len[_id] = link[_id] = pos[_id] = cnt[_id] = 0;
17    return _id++;
18  }
19  void build(string s) {
20    int last = 0; for (int i = 0; i < (int)s.size(); i++) {
21      char c = s[i]; int cur = newnode();
22      len[cur] = len[last] + 1, pos[cur] = i + 1;
23      int p = last;
24      while (~p && !ch[p][c - 'a'])
25        ch[p][c - 'a'] = cur, p = link[p];
26      if (p == -1) link[cur] = 0;
27      else { // conflict: p --> q, p --> cur
28        int q = ch[p][c - 'a'];
29        if (len[p] + 1 == len[q]) { // q is suffix of cur
30          link[cur] = q;
31        } else { // we need auxiliary state
32          int nxt = newnode();
33          len[nxt] = len[p] + 1, link[nxt] = link[q];
34          pos[nxt] = 0;
35          for (int j = 0; j < C; j++)
36            ch[nxt][j] = ch[q][j];
37          while (~p && ch[p][c - 'a'] == q)
38            ch[p][c - 'a'] = nxt, p = link[p];
39          link[q] = link[cur] = nxt;
40        } }
41      cnt[cur]++, last = cur;
42    } }
43  sam;
44  void build_cnt() {
45    vector<int> p(_id); iota(all(p), 0); sort(all(p),
46      [&](int i, int j){ return len[i] > len[j]; });
47    for (int i = 0; i < _id; ++i) if (link[p[i]] != -1)
48      cnt[link[p[i]]] += cnt[p[i]];
49  }
50  bool match(string s) {
51    int p = 0; for (auto& c : s) {
52      if (!ch[p][c - 'a']) return false;
53      p = ch[p][c - 'a'];
54    } return true; }
55}
```

10.7 Minimum Rotation

bb235f

```
1// Author: std_abs
2string rotate(auto s){ int n=s.size(), i=0, j=1;
3  string t = s + s;
4  while (i < n && j < n) {
5    int k = 0; while (k < n && t[i + k] == t[j + k]) ++k;
6    if (t[i + k] <= t[j + k]) j += k + 1; else i += k + 1;
7    if (i == j) ++j;
8  } return t.substr((i < n ? i : j), n);
9}
```

10.8 AC Automaton

285968

```
1// Author: std_abs (slightly modified by Gino)
2// Recap: fail[s] = s 在 trie 上 (也就是某個 pattern 的前綴
  的最長 proper 後綴
3// N := sum(length of all patterns) + 1
4// 1. ac.insert(pat) -> end-node id recorded in ac.pid
5// 2. ac.build_fail()
6// 3. run text: ptr = to[ptr][c-'a'] each char;
  accumulate on your own arrays
7// - pull: rep(i ac.order_cnt-1 ~ 0) val[fail[x]] += val[x]
8// - push: rep(i 0 ~ ac.order_cnt-1) val[x] += val[fail[x]]
9// Tested @yosupo, N <= 1e6, 323ms on building AC
10const int N = 1e6 + 5, C = 26;
11struct AC {
12  int ch[N][C], to[N][C], fail[N], _id, order[N], order_cnt;
13  int np; vector<int> pid; // pattern ID on trie
14  // order[r] := which trie node has BFS order r
15  // fail link tree: (parent) fail[i] <-- i (child)
16  void reset() { _id = np = order_cnt = 0, pid.clear(),
  newnode(); }
```

```

19 int newnode() {
20     fill_n(ch[_id], C, 0), fill_n(to[_id], C, 0);
21     fail[_id] = 0; return _id++;
22 }
23 AC () { reset(); }
24 void insert(string s) {
25     int ptr = 0; for (char c : s) {
26         if (!ch[ptr][c - 'a']) ch[ptr][c - 'a'] = newnode();
27         ptr = ch[ptr][c - 'a'];
28     } np++; pid.emplace_back(ptr);
29 }
30 void build_fail() {
31     queue<int> q; order_cnt = 0;
32     for (int i = 0; i < C; ++i) if (ch[0][i])
33         q.push(ch[0][i]), to[0][i] = ch[0][i];
34     while (!q.empty()) {
35         int u = q.front(); q.pop(); order[order_cnt++] = u;
36         for (int i = 0; i < C; ++i) {
37             if (!ch[u][i]) to[u][i] = to[fail[u]][i];
38             else { int v = ch[u][i], k = fail[u];
39                 while (k && !ch[k][i]) k = fail[k];
40                 if (ch[k][i]) k = ch[k][i];
41                 fail[v] = k, to[u][i] = v;
42                 q.push(v);
43             } } }
44 } ac;

```

10.9 AC Automaton - Dict Links

a62cfa

```

1// Author: Gino, paste this in `struct AC {...}`
2// 3 Modifications to the original AC Automaton:
3// int np; vector<int> pid; bool is_pattern[N];
4// fail[_id] = 0, is_pattern[_id] = false; return _id++;
5// np++; pid.emplace_back(ptr); is_pattern[ptr] = true;
6int dict[N];
7void build_dict() { // call this after build_fail()!
8    for (int i = 0; i < order_cnt; i++) {
9        int u = order[i], f = fail[u];
10        dict[u] = is_pattern[f] ? f : dict[f];
11    }

```

10.10 AC Automaton - Counting Patterns

58fd32

```

1// Author: Gino, paste this in `struct AC {...}`
2ll val[N];
3void count_patterns(string& text, vector<string>& patterns) {
4    for (auto& p : patterns) insert(p); build_fail();
5    ranges::fill(val, 0LL);
6    int ptr = 0; for (auto& c : text) {
7        ptr = to[ptr][c - 'a']; val[ptr]++;
8    }
9    for (int x : span(order, order_cnt) | views::reverse)
10        val[fail[x]] += val[x];
11    for (auto& i : pid) cout << val[i] << endl;
12}

```

10.11 AC Automaton - Pattern Position

輸出每個 pattern 在 text 中第一次出現的位置，若不存在則輸出 -1。
走 AC 自動機時 if (val[ptr] == -1) val[ptr] = i; 接著 pull DP
if (val[x] == -1) continue;
if (val[fail[x]] == -1) val[fail[x]] = val[x];
else val[fail[x]] = min(val[fail[x]], val[x]);

11 Geometry

11.1 Template

acaf6b

```

1using T = long long; // or double
2const double eps = 1e-8;
3int sgn(T x) { return (x > eps) - (x < -eps); }
4struct Pt {
5    T x = 0, y = 0;
6    Pt operator+(Pt a) const { return {x + a.x, y + a.y}; }
7    Pt operator-(Pt a) const { return {x - a.x, y - a.y}; }
8    Pt operator*(T k) const { return {x * k, y * k}; }
9    Pt operator/(T k) const { return {x / k, y / k}; }
10    T operator*(Pt a) const { return x * a.x + y * a.y; }
11    T operator^(Pt a) const { return x * a.y - y * a.x; }
12    bool operator<(Pt a) const {
13        return tie(x, y) < tie(a.x, a.y); }
14    bool operator==(Pt a) const {
15        return !sgn(x - a.x) && !sgn(y - a.y); }
16    friend ostream& operator<<(ostream& o, Pt a)
17    { return o << "(" << a.x << ", " << a.y << ")"; }
18};
19struct Line { Pt a, b; }; // a -> b
20struct Cir { Pt o; double r; };
21T abs2(Pt a) { return a * a; }
22double abs(Pt a) { return sqrt(abs2(a)); }
23int ori(Pt o, Pt a, Pt b) // 1: CCW, -1: CW, 0: collinear
24{ return sgn((a - o) ^ (b - o)); }
25bool onseg(Pt p, Pt a, Pt b) // p on segment ab?
26{ return !ori(p, a, b) && sgn((a - p) * (b - p)) <= 0; }

```

11.2 Basic

8507af

```

1int ud(Pt a) // 0: upper half (incl. +x axis), 1: lower
2{ return !sgn(a.y) ? sgn(a.x) < 0 : a.y < 0; }
3bool cmp(Pt a, Pt b) // polar angle in [0, 2PI)
4{ return ud(a) == ud(b) ? sgn(a ^ b) > 0 : ud(a) < ud(b); }
5bool sameVec(Pt a, Pt b, int d)//d=0: parallel,d=1: same dir
6{ return !sgn(a ^ b) && sgn(a * b) > d * 2 - 2; }
7bool sameVec(Line a, Line b, int d)
8{ return sameVec(a.b - a.a, b.b - b.a, d); }
9Pt rot90(Pt a) { return {-a.y, a.x}; } // CCW
10Pt ref(Pt a) { return ud(a) ? Pt(-a.x, -a.y) : a; }
11T dbarea(vector<Pt>& p) { // doubled signed area, CCW > 0
12    T res = 0; for (int i = 0; i < sz(p); i++)
13        res += p[i] ^ p[(i+1) % sz(p)];
14    return res; }
15// ---- T = double ----
16const double PI = acos(-1);
17double normalize(double x) { // to [0, 2pi)
18    while (x < -eps) x += PI * 2;
19    while (x > PI * 2 + eps) x -= PI * 2;
20    return x; }
21double theta(Pt a) { return normalize(atan2(a.y, a.x)); }
22Pt unit(Pt a) { return a / abs(a); }
23Pt rot(Pt a, double o) { // CCW
24    double c = cos(o), s = sin(o);
25    return Pt(c * a.x - s * a.y, s * a.x + c * a.y); }
26Pt projVec(Pt a, Pt b, Pt c) // vector ac proj to ab
27{ return (b - a) * ((c - a) * (b - a)) / abs2(b - a); }
28Pt projPt(Pt a, Pt b, Pt c) // point c proj to ab
29{ return projVec(a, b, c) + a; }

```

11.3 Line / Segment Intersections

7e8b65

```

1// m=0: segment, m=1: ray from l.a to l.b, m=2: line
2bool banana(Line ls, int ms, Line lt, int mt, int strict) {
3    auto on = [&](Line l, int m, Pt p) {
4        if (ori(l.a, l.b, p) != 0) return false;
5        if (m && abs2(l.a - p) > abs2(l.b - p)) return true;
6        return m == 2 || sgn((p - l.a) * (p - l.b)) <= -strict;
7    };
8    if (sameVec(ls, lt, 0)) {
9        return on(ls, ms, lt.a) || on(ls, ms, lt.b) ||
10            on(lt, mt, ls.a) || on(lt, mt, ls.b);
11    }
12    auto good = [&](Line l, int m, Line o) {
13        if (m && abs((l.a - o.a) ^ (l.a - o.b)) >
14            abs((l.b - o.a) ^ (l.b - o.b))) return true;
15        return m == 2 ||
16            ori(l.a, o.a, o.b) * ori(l.b, o.a, o.b) == -1;
17    };
18    if (good(ls, ms, lt) && good(lt, mt, ls)) return 1;
19    if (!strict) {
20        if (mt != 2 && on(ls, ms, lt.a)) return 1;
21        if (mt == 0 && on(ls, ms, lt.b)) return 1;
22        if (ms != 2 && on(lt, mt, ls.a)) return 1;
23        if (ms == 0 && on(lt, mt, ls.b)) return 1;
24    }
25    return 0;
26}
27// becareful when two lines are parallel
28auto bananaPoint(Line a, Line b) {
29    auto abc = (a.b - a.a) ^ (b.a - a.a);
30    auto abd = (a.b - a.a) ^ (b.b - a.a);
31    return make_pair(b.b * abc - b.a * abd, abc - abd);
32}

```

11.4 Point-Line / Segment Distance

8b23ea

```

1// T = double
2double pointLineDist(Pt p, Line L)
3{ return fabs(((L.b - L.a) ^ (p - L.a)) / abs(L.b - L.a)); }
4double pointSegDist(Pt p, Pt a, Pt b) {
5    if (a == b) return abs(p - a);
6    if (sgn((b - a) * (p - a)) >= 0 &&
7        sgn((a - b) * (p - b)) >= 0)
8        return pointLineDist(p, {a, b});
9    return min(abs(p - a), abs(p - b));
10}

```

11.5 Sort

63db4f

```

1// Dependency: cmp(), btwAngle()
2// cmp in Basic: polar angle sort
3// all points are on line ab. closer to a: front
4bool cmpLine(Pt s, Pt t, Pt a, Pt b) {
5    Pt v = a - b;
6    if (sgn(v.x)) return sgn(s.x - t.x) == sgn(v.x);
7    return sgn(s.y - t.y) == sgn(v.y);
8}
9// intersect points polar angle sort, deno: positive
10bool cmpFractionPolar(pair<Pt, ll> o,
11    pair<Pt, ll> s, pair<Pt, ll> t) {
12    Pt u = s.first * o.second - o.first * s.second;

```

```

13 Pt v = t.first * o.second - o.first * t.second;
14 return cmp(u, v);
15 }
16 struct Seg {
17     Pt a, b; // a.x < b.x
18     bool operator<(const Seg& o) const {
19         if (a == o.a) return ori(o.b, a, b) == 1;
20         if (a.x <= o.a.x) return ori(o.a, a, b) == 1;
21         return ori(a, o.a, o.b) == -1;
22     };
23 struct PolarSeg {
24     Pt a, b; // ori(Pt(0, 0), a, b) > 0
25     bool operator<(const PolarSeg& o) const {
26         if (a == o.a) return ori(o.b, a, b) == -1;
27         if (btwAngle(Pt(0, 0), a, b, o.a, 0))
28             return ori(o.a, a, b) == -1;
29         return ori(a, o.a, o.b) == 1;
30     };
31 struct Arc {
32     // contain(a, b): circle a inside circle b
33     Cir c;
34     int id, s; // 0 -> up, 1 -> down
35     bool operator<(const Arc& b) const {
36         if (id == b.id) return s < b.s;
37         if (contain(c, b.c)) return b.s == 1;
38         if (contain(b.c, c)) return s == 0;
39         if (c.o.y == b.c.o.y) return id < b.id;
40         return c.o.y > b.c.o.y;
41     };

```

11.6 Convex Hull

37fe2b

```

1 auto convexHull(vector<Pt> pts) {
2     sort(all(pts)); vector<Pt> ans = {pts[0]};
3     for (int t = 0; t < 2; t++, reverse(all(pts))) {
4         for (int i = 1, m = sz(ans); i < sz(pts); i++) {
5             while (sz(ans) > m &&
6                 ori(ans[sz(ans) - 2], ans.back(), pts[i]) <= 0)
7                 ans.pop_back();
8             ans.pb(pts[i]); } }
9     if (sz(ans) > 1) ans.pop_back(); return ans; }

```

11.7 Triangle Centers

9c22fc

```

1 // T = double
2 Pt circenter(Pt a, Pt b, Pt c) { // r = abs(o - a)
3     b = b - a, c = c - a; double m = 2 * (b^c);
4     Pt o = {(abs2(b) * c.y - abs2(c) * b.y) / m,
5             (abs2(c) * b.x - abs2(b) * c.x) / m};
6     return o + a;
7 }
8 Pt incenter(Pt a, Pt b, Pt c) { // r = 2 * area / s
9     double x = abs(b - c), y = abs(c - a);
10    double z = abs(a - b), s = x + y + z;
11    return (a*x + b*y + c*z) / s;
12 }
13 Pt centroid(Pt a, Pt b, Pt c) { return (a + b + c) / 3; }
14 Pt orthocenter(Pt a, Pt b, Pt c) {
15     return centroid(a, b, c) * 3 - circenter(a, b, c) * 2;
16 }

```

11.8 Point Inside

109f6b

```

1 // get edge index: check (0, a), (0, b) first
2 // then after binary search, check (a, b)
3 bool inConvex(vector<Pt>& C, Pt p, bool strict = true) {
4     // only works when no three points are collinear
5     int a = 1, b = sz(C) - 1, r = !strict;
6     if (sz(C) == 0) return false;
7     if (sz(C) < 3) return r && onseg(p, C[0], C.back());
8     if (ori(C[0], C[a], C[b]) > 0) swap(a, b);
9     if (ori(C[0], C[a], p) >= r ||
10         ori(C[0], C[b], p) <= -r) return false;
11     while (abs(a - b) > 1) {
12         int c = (a + b) / 2;
13         (ori(C[0], C[c], p) > 0 ? b : a) = c;
14     }
15     return ori(C[a], C[b], p) < r;
16 }
17 // -1: out, 0: edge, 1: in
18 int inPoly(vector<Pt> poly, Pt o, int strict) {
19     int cnt = 0;
20     for (int i = 0; i < sz(poly); i++) {
21         Pt a = poly[i], b = poly[(i + 1) % sz(poly)];
22         if (onseg(o, a, b)) return !strict;
23         cnt ^= ((o.y < a.y) - (o.y < b.y)) * ori(o, a, b) > 0;
24     }
25     return cnt ? 1 : -1;
26 }
27 // relation of q with circumcircle; T = long long
28 bool inCC(array<Pt, 3> p, Pt q) {
29     __int128 det = 0;
30     for (int i = 0; i < 3; i++)

```

```

31     det += __int128(abs2(p[i]) - abs2(q)) *
32         ((p[(i + 1) % 3] - q) ^ (p[(i + 2) % 3] - q));
33     return det > 0; // in: >0, on: =0, out: <0
34 }

```

11.9 Point In Triangle

144fc0

```

1 // number of p with a < p < b and ori(p, a, b) < 0
2 int under(Pt a, Pt b) {}
3 // number of points with a < p < b and ori(p, a, b) = 0
4 int edge(Pt a, Pt b) {}
5 // check if this number is calculated
6 bool check(Pt p) {}
7 // number of points that strictly inside the triangle
8 int inTri(array<Pt, 3> arr) {
9     sort(all(arr)); auto [a, b, c] = arr; intx = ori(b, a, c);
10    if (x == 0) return 0;
11    if (x == 1) return under(a, b) + under(b, c) -
12        under(a, c) - edge(a, c);
13    return under(a, c) - under(a, b) -
14        under(b, c) - edge(a, b) - edge(b, c) - check(b);
15 }

```

11.10 Vector In Polygon

1b2dce

```

1 // ori(a, b, c) >= 0, valid: "strict" angle from a-b to a-c
2 bool btwAngle(Pt a, Pt b, Pt c, Pt p, int strict) {
3     return ori(a, b, p) >= strict &&
4         ori(a, p, c) >= strict;
5 }
6 // whether vector{cur, p} is in order prv, cur, nxt
7 bool inside(Pt prv, Pt cur, Pt nxt, Pt p, int strict) {
8     if (ori(cur, nxt, prv) >= 0)
9         return btwAngle(cur, nxt, prv, p, strict);
10    return !btwAngle(cur, prv, nxt, p, !strict);
11 }

```

11.11 Polygon Cut

c08d26

```

1 // Dependency: bananaPoint(); T = double
2 // Cut a convex polygon with line (s -> e), keep right side
3 vector<Pt> polyCut(vector<Pt> poly, Pt s, Pt e) {
4     vector<Pt> res; for (int i = 0; i < sz(poly); i++) {
5         Pt cur = poly[i], prv = i ? poly[i - 1] : poly.back();
6         bool side = ori(s, e, cur) < 0;
7         if (side != (ori(s, e, prv) < 0)) {
8             auto [p, d] = bananaPoint({s, e}, {cur, prv});
9             res.pb(p / d);
10        } if (side) res.pb(cur);
11    } return res;
12 }

```

11.12 Half-Plane Intersection

c71070

```

1 // Dependency: sameVec(), cmp(); T = long long
2 /* Having solution, check size > 2 */
3 /* --^-- Line.a --^-- Line.b --^-- */
4 auto halfPlaneIntersection(vector<Line> arr) {
5     auto areaPair = [&](Line a, Line b) {
6         return make_pair((a.b - a.a) ^ (b.a - a.a),
7                         (a.b - a.a) ^ (b.b - a.a));
8     };
9     auto isin = [&](Line l0, Line l1, Line l2) {
10        // Check inter(l1, l2) strictly in l0
11        auto [a02X, a02Y] = areaPair(l0, l2);
12        auto [a12X, a12Y] = areaPair(l1, l2);
13        if (a12X - a12Y < 0) a12X *= -1, a12Y *= -1;
14        return (__int128)a02Y * a12X -
15            (__int128)a02X * a12Y > 0; // C^4
16    };
17    sort(all(arr), [&](Line a, Line b) {
18        if (sameVec(a, b, 1))
19            return ori(a.a, a.b, b.b) < 0;
20        return cmp(a.b - a.a, b.b - b.a);
21    });
22    deque<Line> dq(1, arr[0]);
23    auto popBack = [&](int t, Line p) {
24        while (sz(dq) >= t &&
25            !isin(p, dq[sz(dq) - 2], dq.back()))
26            dq.pop_back();
27    };
28    auto popFront = [&](int t, Line p) {
29        while (sz(dq) >= t && !isin(p, dq[0], dq[1]))
30            dq.pop_front();
31    };
32    for (auto p : arr) if (!sameVec(dq.back(), p, 1))
33        popBack(2, p), popFront(2, p), dq.pb(p);
34    popBack(3, dq[0]), popFront(3, dq.back());
35    return vector<Line>(all(dq));
36 }

```

11.13 Cyclic Ternary Search

e3165d

```

1 /* bool pred(int a, int b);
2 f(0) ~ f(n - 1) is a cyclic-shift U-function
3 return idx s.t. pred(x, idx) is false forall x*/

```

```

4 int cycTsearch(int n, auto pred) {
5   if (n == 1) return 0;
6   int l = 0, r = n; bool rv = pred(1, 0);
7   while (r - l > 1) {
8     int m = (l + r) / 2;
9     if (pred(0, m) ? rv : pred(m, (m + 1) % n)) r = m;
10    else l = m;
11   } return pred(l, r % n) ? l : r % n;
12 }

```

11.14 Circle Intersections

734f65

```

1 // Dependency: cycTsearch(); T = double for circles
2 vector<Pt> circleLineIntersect(Cir c, Line l) {
3   Pt p = l.a + (l.b - l.a) *
4     ((c.o - l.a) * (l.b - l.a)) / abs2(l.b - l.a);
5   double s = (l.b - l.a) ^ (c.o - l.a);
6   double h2 = c.r * c.r - s * s / abs2(l.b - l.a);
7   if (sgn(h2) == -1) return {};
8   if (sgn(h2) == 0) return {p};
9   Pt h = (l.b - l.a) / abs(l.b - l.a) * sqrt(h2);
10  return {p - h, p + h};
11 }
12 // covered area of c1: arc from res[0] to res[1], CCW
13 vector<Pt> circlesIntersect(Cir c1, Cir c2) {
14   double d2 = abs2(c1.o - c2.o), d = sqrt(d2);
15   if (d < max(c1.r, c2.r) - min(c1.r, c2.r) ||
16       d > c1.r + c2.r) return {};
17   Pt u = (c1.o + c2.o) / 2 +
18     (c1.o - c2.o) *
19     ((c2.r * c2.r - c1.r * c1.r) / (2 * d2));
20   double A = sqrt((c1.r + c2.r + d) *
21     (c1.r - c2.r + d) * (c1.r + c2.r - d) *
22     (-c1.r + c2.r + d));
23   Pt v = rot90(c2.o - c1.o) * A / (2 * d2);
24   if (sgn(v.x) == 0 && sgn(v.y) == 0) return {u};
25   return {u - v, u + v};
26 }
27 vector<pii> convexLineIntersect(vector<Pt>& C, Line la) {
28   auto dis = [&](int p) {
29     return (la.b - la.a) ^ (C[p] - la.a);
30   };
31   auto gao = [&](int s) {
32     return cycTsearch(sz(C), [&](int i, int j) {
33       return sgn(dis(i) - dis(j)) == s;
34     });
35   };
36   int x = gao(1), y = gao(-1), n = sz(C);
37   if (sgn(dis(x)) < 0 || sgn(dis(y)) > 0) return {};
38   if (sgn(dis(x)) == 0 || sgn(dis(y)) == 0) {
39     int v = ((sgn(dis(x)) == 0 ? x : y) + n - 1) % n;
40     vector<pii> vec;
41     for (int i = 0; i < 3; i++, v = (v + 1) % n)
42       if (sgn(dis(v)) == 0) vec.emplace_back(v, v);
43     return vec;
44   }
45   auto get = [&](int l, int r, int s) {
46     while ((l + 1) % n != r) {
47       int m = ((l + r + (l < r ? 0 : n)) / 2) % n;
48       if (sgn(dis(m)) == s ? l : r) = m;
49     }
50     if (sgn(dis(r)) == 0) return pii(r, r);
51     return pii(l, r);
52   };
53   return {get(x, y, 1), get(y, x, -1)};
54 }

```

11.15 Tangents

8def22

```

1 // Dependency: cycTsearch(); T = double for circle tangents
2 auto circlePointTangent(Cir c, Pt p) {
3   vector<Line> res;
4   double dSq = abs2(p - c.o);
5   if (sgn(dSq - c.r * c.r) == 0) {
6     res.push_back({p, p + rot90(p - c.o)});
7   } else if (dSq > c.r * c.r) {
8     double s = dSq - c.r * c.r;
9     Pt v = p + (c.o - p) * s / dSq;
10    Pt u = rot90(c.o - p) * sqrt(s) * c.r / dSq;
11    res.push_back({p, v + u});
12    res.push_back({p, v - u});
13  }
14  return res;
15 }
16 auto circlesTangent(Cir c1, Cir c2, int sign1) {
17   // sign1 = 1 for outer tang, -1 for inter tang
18   vector<Line> res;
19   double dSq = abs2(c1.o - c2.o);
20   if (sgn(dSq) == 0) return res;
21   double d = sqrt(dSq);
22   Pt v = (c2.o - c1.o) / d;

```

```

23   double c = (c1.r - sign1 * c2.r) / d;
24   if (c * c > 1) return res;
25   double h = sqrt(max(0.0, 1.0 - c * c));
26   for (int sign2 = 1; sign2 >= -1; sign2 -= 2) {
27     Pt n = Pt(v.x * c - sign2 * h * v.y,
28       v.y * c + sign2 * h * v.x);
29     Pt p1 = c1.o + n * c1.r;
30     Pt p2 = c2.o + n * (c2.r * sign1);
31     if (sgn(p1.x - p2.x) == 0 &&
32         sgn(p1.y - p2.y) == 0)
33       p2 = p1 + rot90(c2.o - c1.o);
34     res.push_back({p1, p2});
35   }
36   return res;
37 }
38 /* The point should be strictly out of hull
39    return arbitrary point on the tangent line */
40 pii pointConvexTangent(vector<Pt>& C, Pt p) {
41   auto gao = [&](int s) {
42     return cycTsearch(sz(C), [&](int x, int y) {
43       return ori(p, C[x], C[y]) == s;
44     });
45   };
46   return pii(gao(1), gao(-1));
47 } // return (a, b), ori(p, C[a], C[b]) >= 0

```

11.16 Bisector

5348b9

```

1 Pt bisector(Pt a, Pt b, Pt c) { Pt s = b - a, t = c - a;
2   t = t * abs(s) / abs(t); return s + t; }

```

11.17 Minimum Enclosing Circle

04b498

```

1 // Dependency: circenter(); T = double
2 Cir minEnclosingCircle(vector<Pt> p) {
3   auto seed = chrono::steady_clock::now().
4     time_since_epoch().count();
5   shuffle(all(p), mt19937(seed));
6   double r = 0.0; Pt cent = p[0];
7   for (int i = 1; i < sz(p); i++) {
8     if (abs2(cent - p[i]) <= r) continue;
9     cent = p[i]; r = 0.0;
10    for (int j = 0; j < i; j++) {
11      if (abs2(cent - p[j]) <= r) continue;
12      cent = (p[i] + p[j]) / 2; r = abs2(p[j] - cent);
13      for (int k = 0; k < j; k++) {
14        if (abs2(cent - p[k]) <= r) continue;
15        cent = circenter(p[i], p[j], p[k]);
16        r = abs2(p[k] - cent);
17      }
18    }
19    return {cent, sqrt(r)};
20 }

```

11.18 Minkowski Sum

fdd915

```

1 // Dependency: convexHull()
2 auto minkowski(vector<Pt> a, vector<Pt> b) {
3   a = convexHull(a), b = convexHull(b);
4   int n = sz(a), m = sz(b);
5   vector<Pt> c = {a[0] + b[0]}, s1, s2;
6   for (int i = 0; i < n; i++) s1.pb(a[(i+1) % n] - a[i]);
7   for (int i = 0; i < m; i++) s2.pb(b[(i+1) % m] - b[i]);
8   for (int p1 = 0, p2 = 0; p1 < n || p2 < m;)
9     if (p2 == m ||
10        (p1 < n && sgn(s1[p1] ^ s2[p2]) >= 0))
11       c.pb(c.back() + s1[p1++]);
12     else
13       c.pb(c.back() + s2[p2++]);
14   return convexHull(c);
15 }

```

11.19 Convex Hull Distance

f7df29

```

1 // Dependency: minkowski(), inConvex(), pointSegDist()
2 // T = double
3 double convexHullDist(vector<Pt> A, vector<Pt> B) {
4   Pt 0(0, 0); for (auto& p : B) p = 0 - p;
5   auto C = minkowski(A, B);
6   if (inConvex(C, 0)) return 0;
7   double ans = pointSegDist(0, C.back(), C[0]);
8   for (int i = 0; i + 1 < sz(C); i++)
9     ans = min(ans, pointSegDist(0, C[i], C[i + 1]));
10  return ans; }

```

11.20 Closest Point Pair

ad1bba

```

1 // returns {min squared dist, {i, j}}, n >= 2, |coord| < 1e9
2 pair<T, pair<int, int>> closestPair(vector<Pt> p) {
3   vector<int> id(sz(p)); iota(all(id), 0);
4   sort(all(id),
5     [&](int a, int b) { return p[a].y < p[b].y; });
6   set<pair<Pt, int>> s; // (point, id)
7   pair<T, pair<int, int>> best = {(T)9e18, {-1, -1}};
8   int l = 0; for (int i : id) {
9     if (!best.first) break;
10    T d = sqrtl(best.first) + is_integral_v<T>;
11    while (p[id[l]].y <= p[i].y - d)

```

```

12     s.erase({p[id[l]], id[l]}), l++;
13     auto lo = s.lower_bound({p[i] - Pt{d, 0}, 0});
14     auto hi = s.upper_bound({p[i] + Pt{d, 0}, sz(p)});
15     for (; lo != hi; lo++) {
16         auto [q, j] = *lo;
17         best = min(best,
18             {abs2(q - p[i]), {min(i, j), max(i, j)}});
19     } s.insert({p[i], i});
20 } return best;
21 }

```

11.21 Polygon-Circle Intersection

18454b

```

1 // T = double
2 double areaSector(Pt pa, Pt pb, double r) {
3     if (abs(pa) < abs(pb)) swap(pa, pb);
4     if (abs(pb) < eps) return 0;
5     double S, h, t;
6     double a = abs(pb), b = abs(pa), c = abs(pb - pa);
7     double cosB = pb * (pb - pa) / a / c;
8     double B = acos(cosB);
9     double cosC = pa * pb / a / b, C = acos(cosC);
10    if (a > r) {
11        S = C / 2 * r * r;
12        h = a * b * sin(C) / c;
13        if (h < r && B < PI / 2)
14            S -= acos(h / r) * r * r - h * sqrt(r * r - h * h);
15    } else if (b > r) {
16        t = PI - B - asin(sin(B) / r * a);
17        S = 0.5 * a * r * sin(t) + (C - t) / 2 * r * r;
18    } else {
19        S = 0.5 * sin(C) * a * b;
20    }
21    return S;
22 }
23 double areaPolyCircle(vector<Pt> poly, Pt 0, double r) {
24     double S = 0;
25     int n = sz(poly);
26     for (int i = 0; i < n; i++)
27         S += areaSector(poly[i] - 0,
28             poly[(i + 1) % n] - 0, r) *
29             ori(0, poly[i], poly[(i + 1) % n]);
30     return fabs(S);
31 }

```

11.22 Union of Polygons

9d7265

```

1 // Dependency: sameVec(), bananaPoint(), cmpLine()
2 // in CCW order, use index as tiebreaker when collinear
3 auto polysBorder(vector<vector<Pt>> poly, int id) {
4     auto get = [&](auto& p, int i) {
5         return make_pair(p[i], p[(i + 1) % sz(p)]);
6     };
7     vector<pair<Pt, Pt>> seg;
8     for (int e = 0; e < sz(poly[id]); e++) {
9         auto [s, t] = get(poly[id], e);
10        vector<pair<Pt, int>> vec;
11        vec.emplace_back(s, -1 << 30);
12        vec.emplace_back(t, 1 << 30);
13        for (int i = 0; i < sz(poly); i++) {
14            int st = find_if(poly[i].begin(), poly[i].end(),
15                [&](Pt p) { return ori(p, s, t) == 1; }) -
16                poly[i].begin();
17            if (st == sz(poly[i])) continue;
18            for (int j = st; j < st + sz(poly[i]); j++) {
19                auto [a, b] = get(poly[i], j % sz(poly[i]));
20                if (sameVec(a - b, s - t, -1)) {
21                    if (ori(a, b, s) == 0 &&
22                        sameVec(a - b, s - t, 1) && i <= id) {
23                        vec.emplace_back(a, -1);
24                        vec.emplace_back(b, 1);
25                    }
26                } else {
27                    int s1 = ori(a, s, t) == 1;
28                    int s2 = ori(b, s, t) == 1;
29                    if (s1 ^ s2) {
30                        auto [p, d] = bananaPoint({a, b}, {s, t});
31                        vec.emplace_back(p / d, s1 ? 1 : -1);
32                    }
33                }
34            }
35            sort(all(vec), [&](auto i, auto j) {
36                return cmpLine(i.first, j.first, s, t);
37            });
38            int base = 1 << 30;
39            Pt lst(0, 0);
40            for (auto [cur, val] : vec) {
41                if (!base) seg.emplace_back(lst, cur);
42                lst = cur, base += val;
43            }
44            return seg;
45        }
46    }
47    double polysUnionArea(vector<vector<Pt>> poly) {

```

```

45    double res = 0;
46    for (int i = 0; i < sz(poly); i++) {
47        auto seg = polysBorder(poly, i);
48        for (auto [l, r] : seg) res += l ^ r;
49    } return res / 2;
50 }

```

11.23 Rotating Sweep Line

769b9b

```

1 // Dependency: ref()
2 struct Event {
3     Pt d; int u, v;
4     bool operator<(const Event& o) const {
5         return sgn(d ^ o.d) > 0;
6     };
7 void rotatingSweepLine(vector<Pt> pts) {
8     int n = sz(pts);
9     vector<int> ord(n), pos(n); vector<Event> e;
10    for (int i = 0; i < n; i++)
11        for (int j = i + 1; j < n; j++)
12            e.push_back({ref(pts[i] - pts[j]), i, j});
13    sort(all(e)); iota(all(ord), 0);
14    sort(all(ord), [&](int i, int j) {
15        return sgn(pts[i].y - pts[j].y) == 0 ?
16            pts[i].x < pts[j].x : pts[i].y < pts[j].y;
17    });
18    for (int i = 0; i < n; i++) pos[ord[i]] = i;
19    auto makeReverse = [](auto v) {
20        sort(all(v));
21        v.resize(unique(all(v)) - v.begin());
22        vector<pii> segs;
23        for (int i = 0, j = 0; i < sz(v); i = j) {
24            for (; j < sz(v) && v[j] - v[i] <= j - i; j++);
25            segs.emplace_back(v[i], v[j - 1] + 1 + 1);
26        }
27        return segs;
28    };
29    for (int i = 0, j = 0; i < sz(e); i = j) {
30        vector<int> tmp;
31        for (; j < sz(e) && !(e[i] < e[j]); j++)
32            tmp.pb(min(pos[e[j].u], pos[e[j].v]));
33        for (auto [l, r] : makeReverse(tmp)) {
34            reverse(ord.begin() + l, ord.begin() + r);
35            for (int t = l; t < r; t++) pos[ord[t]] = t;
36            // update value here
37        }
38    }

```

11.24 Pick's Theorem

Consider a polygon which vertices are all lattice points.
 Let i = number of points inside the polygon.
 Let b = number of points on the boundary of the polygon.
 Then we have the following formula:

$$\text{Area} = i + b/2 - 1$$

12 Number Theory

12.1 Mod Sum

60beb8

```

1 /**
2  * Author: Simon Lindholm
3  * Date: 2015-06-23
4  * License: CC0
5  * Source: own work
6  * Description: Sums of mod'ed arithmetic progressions.
7  *
8  * \texttt{modsum(to, c, k, m)} =  $\sum_{i=0}^{\text{to}-1} \{(ki+c) \% m\}$ .
9  * \texttt{divsum} is similar but for floored division.
10 * Time:  $\mathcal{O}(\log(m))$ , with a large constant.
11 * Status: Tested for all  $|k|, |c|, \text{to}, m \leq 50$ , and on
12 kattis:aladin
13 */
14 typedef unsigned long long ull;
15 ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
16 /// ^ written in a weird way to deal with overflows
17 /// correctly
18 ull divsum(ull to, ull c, ull k, ull m) {
19     ull res = k / m * sumsq(to) + c / m * to;
20     k %= m; c %= m;
21     if (!k) return res;
22     ull to2 = (to * k + c) / m;
23     return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
24 }
25 ll modsum(ull to, ll c, ll k, ll m) {
26     c = ((c % m) + m) % m;
27     k = ((k % m) + m) % m;
28     return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
29 }

```

12.2 Extended Lucas Theorem

3bed10

```

1/**
2 * Author: koukanni
3 * Date: 2025
4 * License: CC0
5 * Source:
6 * Description: Computes  $C(n, m) \bmod P$  where  $P$  is not
7 * necessarily prime
8 * Functions
9 * exgcd(a, b, x, y): Extended Euclidean algorithm
10 * inv(a, p): Modular inverse of a modulo p
11 * qpow(a, b, p): Fast modular exponentiation
12 * CRT(n, a, m): Chinese Remainder Theorem
13 * calc(n, x, P): Helper function for computing factorial
14 * with prime power
15 * multilucas(n, m, x, P): Lucas theorem for prime power
16 * modulus
17 * exlucas(n, m, P): Main function - Extended Lucas theorem
18 * usage: exlucas(n, m, P) returns  $C(n, m) \bmod P$ 
19 * Time:  $O(P \log(P))$ 
20 * Status: tested
21 */
22 typedef long long ll;
23 void exgcd(ll a, ll b, ll& x, ll& y) {
24     if (!b)
25         x = 1, y = 0;
26     else
27         exgcd(b, a % b, y, x), y -= a / b * x;
28 }
29 ll inv(ll a, ll p) {
30     ll x, y;
31     exgcd(a, p, x, y);
32     return (x + p) % p;
33 }
34 ll qpow(ll a, ll b, ll p) {
35     ll r = 1;
36     for (; b; b >>= 1, a = a * a % p)
37         if (b & 1) r = r * a % p;
38     return r;
39 }
40 ll CRT(int n, ll* a, ll* m) {
41     ll M = 1, p = 0;
42     for (int i = 1; i <= n; i++) M = M * m[i];
43     for (int i = 1; i <= n; i++) {
44         ll w = M / m[i], x, y;
45         exgcd(w, m[i], x, y);
46         p = (p + a[i] * w * x % M) % M;
47     }
48     return (p % M + M) % M;
49 }
50 ll calc(ll n, ll x, ll P) {
51     if (!n) return 1;
52     ll s = 1;
53     for (ll i = 1; i <= P; i++)
54         if (i % x) s = s * i % P;
55     s = qpow(s, n / P, P);
56     for (ll i = n / P * P + 1; i <= n; i++)
57         if (i % x) s = i % P * s % P;
58     return s * calc(n / x, x, P) % P;
59 }
60 ll multilucas(ll n, ll m, ll x, ll P) {
61     if (n < m) return 0;
62     int cnt = 0;
63     for (ll i = n; i; i /= x) cnt += i / x;
64     for (ll i = m; i; i /= x) cnt -= i / x;
65     for (ll i = n - m; i; i /= x) cnt -= i / x;
66     return qpow(x, cnt, P) % P * calc(n, x, P) % P *
67         inv(calc(n - m, x, P), P) % P;
68 }
69 ll exlucas(ll n, ll m, ll P) {
70     int cnt = 0;
71     ll p[20], a[20];
72     for (ll i = 2; i * i <= P; i++) {
73         if (P % i == 0) {
74             p[++cnt] = i;
75             while (P % i == 0) p[cnt] = p[cnt] * i, P /= i;
76             a[cnt] = multilucas(n, m, i, p[cnt]);
77         }
78     }
79     if (P > 1) p[++cnt] = P, a[cnt] = multilucas(n, m, P, P);
80     return CRT(cnt, a, p);
81 }

```

12.3 Prime Sieve and Defactor

e2350f

```

1// Author: Gino
2const int maxc = 1e6 + 1;
3vector<int> lpf;
4vector<int> prime;

```

```

5
6void sieve() {
7    prime.clear();
8    lpf.resize(maxc, 1);
9    for (int i = 2; i < maxc; i++) {
10        if (lpf[i] == 1) {
11            lpf[i] = i;
12            prime.emplace_back(i);
13        }
14        for (auto& j : prime) {
15            if (i * j >= maxc) break;
16            lpf[i * j] = j;
17            if (j == lpf[i]) break;
18        }
19    }
20    vector<pii> fac;
21    void defactor(int u) {
22        fac.clear();
23        while (u > 1) {
24            int d = lpf[u];
25            fac.emplace_back(make_pair(d, 0));
26            while (u % d == 0) {
27                u /= d;
28                fac.back().second++;
29            }
30        }
31    }
32 }

```

12.4 Harmonic Series

7c56cc

```

1// Author: Gino
2//  $O(n \log n)$ 
3for (int i = 1; i <= n; i++) {
4    for (int j = i; j <= n; j += i) {
5        //  $O(1)$  code
6    }
7}
8
9// PIE
10// given array a[0], a[1], ..., a[n - 1]
11// calculate  $dp[x]$  = number of pairs (a[i], a[j]) such that
12//  $\gcd(a[i], a[j]) = x$  // ( $i < j$ )
13//
14// idea: let  $mc(x)$  = # of y s.t.  $x|y$ 
15//  $f(x)$  = # of pairs s.t.  $\gcd(a[i], a[j]) \geq x$ 
16//  $f(x) = C(mc(x), 2)$ 
17//  $dp[x] = f(x) - \sum(dp[y], x < y \text{ and } x|y)$ 
18const int maxc = 1e6;
19vector<int> cnt(maxc + 1, 0), dp(maxc + 1, 0);
20for (int i = 0; i < n; i++)
21    cnt[a[i]]++;
22
23for (int x = maxc; x >= 1; x--) {
24    ll cnt_mul = 0; // number of multiples of x
25    for (int y = x; y <= maxc; y += x)
26        cnt_mul += cnt[y];
27
28    dp[x] = cnt_mul * (cnt_mul - 1) / 2; // number of pairs
29    // that are divisible by x
30    for (int y = x + x; y <= maxc; y += x)
31        dp[x] -= dp[y]; // PIE: subtract all dp[y] for y >
32    // x and x|y
33}

```

12.5 Count Number of Divisors

0a5616

```

1// Author: Gino
2// Function: Count the number of divisors for all  $x \leq 10^6$ 
3// using harmonic series
4const int maxc = 1e6;
5vector<int> facs;
6
7void find_all_divisors() {
8    facs.clear(); facs.resize(maxc + 1, 0);
9    for (int x = 1; x <= maxc; x++) {
10        for (int y = x; y <= maxc; y += x) {
11            facs[y]++;
12        }
13    }
14}

```

12.6 數論分塊

cd4e6b

```

1// Author: Gino
2/*
3n = 17
4 i: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
5 n/i: 17 8 5 4 3 2 2 2 1 1 1 1 1 1 1 1 1
6           ^       ^
7           L(2)   R(2)
8
9 L(x) := left bound for  $n/i = x$ 
10 R(x) := right bound for  $n/i = x$ 
11
12 ===== FORMULA =====
13 >>>  $R = n / (n/L) <<<$ 
14 =====

```

```

15
16 Example: L(2) = 6
17           R(2) = 17 / (17 / 6)
18               = 17 / 2
19               = 8
20 */
21 // ===== CODE =====
22
23 for (ll l = 1, r = 1, q = n; l <= n; l = r + 1) {
24     q = n/l;
25     r = n/q;
26     // Process your code here
27 }
28 // q, l, r: 17 1 1
29 // q, l, r: 8 2 2
30 // q, l, r: 5 3 3
31 // q, l, r: 4 4 4
32 // q, l, r: 3 5 5
33 // q, l, r: 2 6 8
34 // q, l, r: 1 9 17

```

12.7 Pollard's rho

```

1 // Author: Unknown
2 // Function: Find a non-trivial factor of a big number in
  O(n^(1/4) log^2(n))
3
4 ll find_factor(ll number) {
5     __int128 x = 2;
6     for (__int128 cycle = 1; ; cycle++) {
7         __int128 y = x;
8         for (int i = 0; i < (1<<cycle); i++) {
9             x = (x * x + 1) % number;
10            __int128 factor = __gcd(x - y, number);
11            if (factor > 1)
12                return factor;
13        }
14    }
15 }
16
17 # Author: Unknown
18 # Function: Find a non-trivial factor of a big number in
  O(n^(1/4) log^2(n))
19
20 from itertools import count
21 from math import gcd
22 from sys import stdin
23
24 for s in stdin:
25     number, x = int(s), 2
26     brk = False
27     for cycle in count(1):
28         y = x
29         if brk:
30             break
31         for i in range(1<< cycle):
32             x = (x * x + 1) % number
33             factor = gcd(x - y, number)
34             if factor > 1:
35                 print(factor)
36                 brk = True
37                 break

```

12.8 Miller Rabin

686187

```

1 // Author: Unknown, Modified by Gino
2 // Function: Check if a number is a prime in O(100 *
  log^2(n))
3 // miller_rabin(): return 1 if prime, 0 otherwise
4 inline ll mul(ll x, ll y, ll mod) {
5     return (__int128)(x) * y % mod;
6 }
7 ll mypow(ll a, ll b, ll mod) {
8     ll r = 1;
9     while (b > 0) {
10        if (b & 1) r = mul(r, a, mod);
11        a = mul(a, a, mod);
12        b >>= 1;
13    }
14    return r;
15 }
16 bool witness(ll a, ll n, ll u, int t) {
17     ll x = mypow(a, u, n);
18     for (int i = 0; i < t; i++) {
19         ll nx = mul(x, x, n);
20         if (nx == 1 && x != 1 && x != n-1) return true;
21         x = nx;
22     }
23     return x != 1;
24 }
25 bool miller_rabin(ll n) {
26     // if n >= 3,474,749,660,383
27     // change {2, 3, 5, 7, 11, 13} to

```

```

28 // {2, 325, 9375, 28178, 450775, 9780504, 1795265022}
29 if (n < 2) return false;
30 if (!(n & 1)) return n == 2;
31 ll u = n - 1; int t = 0;
32 while (!(u & 1)) u >>= 1, t++;
33 for (ll a : {2, 3, 5, 7, 11, 13}) {
34     if (a % n == 0) continue;
35     if (witness(a, n, u, t)) return false;
36 }
37 return true;
38 }
39 // bases that make sure no pseudoprimes flee from test
40 // if WA, replace randll(n - 1) with these bases:
41 // n < 4,759,123,141      3 : 2, 7, 61
42 // n < 1,122,004,669,633  4 : 2, 13, 23, 1662803
43 // n < 3,474,749,660,383  6 : pirms <= 13
44 // n < 2^64              7 :
45 // 2, 325, 9375, 28178, 450775, 9780504, 1795265022

```

12.9 Discrete Log

a04e88

```

1 // Author: ckiseki
2 // find x^? = y (mod M)
3 // O(sqrt(M))
4 template<typename Int>
5 Int BSGS(Int x, Int y, Int M) {
6     // x^? \equiv y (mod M)
7     Int t = 1, c = 0, g = 1;
8     for (Int M_ = M; M_ > 0; M_ >>= 1) g = g * x % M;
9     for (g = gcd(g, M); t % g != 0; ++c) {
10        if (t == y) return c;
11        t = t * x % M;
12    }
13    if (y % g != 0) return -1;
14    t /= g, y /= g, M /= g;
15    Int h = 0, gs = 1;
16    for (; h * h < M; ++h) gs = gs * x % M;
17    unordered_map<Int, Int> bs;
18    for (Int s = 0; s < h; bs[y] = ++s) y = y * x % M;
19    for (Int s = 0; s < M; s += h) {
20        t = t * gs % M;
21        if (bs.count(t)) return c + s + h - bs[t];
22    }
23    return -1;
24 }

```

12.10 Discrete Sqrt

3f16e1

```

1 // Author: ckiseki
2 // find solution for x^2 = n (mod P)
3 // O(log^2 P)
4 int get_root(int n, int P) { // ensure 0 <= n < P
5     if (P == 2 or n == 0) return n;
6     auto check = [&](ll d) {
7         return modpow(int(x), (P - 1) / 2, P);
8     };
9     if (check(n) != 1) return -1;
10    mt19937 rnd(7122); ll d = 1, w;
11    while (check(w = (z * z - n + P) % P) != P - 1)
12        z = rnd() % P;
13    const auto M = [P, w](auto &u, auto &v) {
14        auto [a, b] = u; auto [c, d] = v;
15        return make_pair((a * c + b * d % P * w) % P,
16            (a * d + b * c) % P);
17    };
18    pair<ll, ll> r(1, 0), e(z, 1);
19    for (int q = (P + 1) / 2; q; q >>= 1, e = M(e, e))
20        if (q & 1) r = M(r, e);
21    return int(r.first); // sqrt(n) mod P where P is prime

```

12.11 Fast Power

Note: $a^n \equiv a^{(n \bmod (p-1))} \pmod{p}$

12.12 Extend GCD

50b7fc

```

1 // Author: Gino
2 // [Usage]
3 // bezout(a, b, c):
4 //     find solution to ax + by = c
5 //     return {-LINF, -LINF} if no solution
6 // inv(a, p):
7 //     find modulo inverse of a under p
8 //     return -1 if not exist
9 // CRT(vector<ll>& a, vector<ll>& m)
10 //     find a solution pair (x, mod) satisfies all x = a[i]
  (mod m[i])
11 //     return {-LINF, -LINF} if no solution
12
13 const ll LINF = 4e18;
14 typedef pair<ll, ll> pll;
15 template<typename T1, typename T2>
16 T1 chmod(T1 a, T2 m) {
17     return (a % m + m) % m;
18 }

```

```

19
20 ll GCD;
21 pll extgcd(ll a, ll b) {
22     if (b == 0) {
23         GCD = a;
24         return pll{1, 0};
25     }
26     pll ans = extgcd(b, a % b);
27     return pll{ans.second, ans.first - a/b * ans.second};
28 }
29 pll bezout(ll a, ll b, ll c) {
30     bool negx = (a < 0), negy = (b < 0);
31     pll ans = extgcd(abs(a), abs(b));
32     if (c % GCD != 0) return pll{-LINF, -LINF};
33     return pll{ans.first * c/GCD * (negx ? -1 : 1),
34               ans.second * c/GCD * (negy ? -1 : 1)};
35 }
36 ll inv(ll a, ll p) {
37     if (p == 1) return -1;
38     pll ans = bezout(a % p, -p, 1);
39     if (ans == pll{-LINF, -LINF}) return -1;
40     return chmod(ans.first, p);
41 }
42 pll CRT(vector<ll>& a, vector<ll>& m) {
43     for (int i = 0; i < (int)a.size(); i++)
44         a[i] = chmod(a[i], m[i]);
45
46     ll x = a[0], mod = m[0];
47     for (int i = 1; i < (int)a.size(); i++) {
48         pll sol = bezout(mod, m[i], a[i] - x);
49         if (sol.first == -LINF) return pll{-LINF, -LINF};
50
51         // prevent long long overflow
52         ll p = chmod(sol.first, m[i] / GCD);
53         ll lcm = mod / GCD * m[i];
54         x = chmod((__int128)p * mod + x, lcm);
55         mod = lcm;
56     }
57     return pll{x, mod};
58 }

```

12.13 Mu + Phi

ab4a91

```

1 // Author: Gino
2 const int maxn = 1e6 + 5;
3 ll f[maxn];
4 vector<int> lpf, prime;
5 void build() {
6     lpf.clear(); lpf.resize(maxn, 1);
7     prime.clear();
8     f[1] = ...; /* mu[1] = 1, phi[1] = 1 */
9     for (int i = 2; i < maxn; i++) {
10         if (lpf[i] == 1) {
11             lpf[i] = i; prime.emplace_back(i);
12             f[i] = ...; /* mu[i] = 1, phi[i] = i-1 */
13         }
14         for (auto& j : prime) {
15             if (i*j >= maxn) break;
16             lpf[i*j] = j;
17             if (i % j == 0) f[i*j] = ...; /* 0, phi[i*j] */
18             else f[i*j] = ...; /* -mu[i], phi[i]*phi[j] */
19             if (j >= lpf[i]) break;
20         }
21     }
22 }

```

12.14 Other Formulas

- Pisano Period: 任何線性遞迴（比如費氏數列）模任何一個數字 M 都會循環，找循環節 $\pi(M)$ 先質因數分解 $M = \prod p_i^{e_i}$ ，然後 $\pi(M) = \text{lcm}(\pi(p_i^{e_i}))$ ，
- Inversion: $aa^{-1} \equiv 1 \pmod{m}$. a^{-1} exists iff $\gcd(a, m) = 1$.
- Linear inversion: $a^{-1} \equiv (m - \lfloor \frac{m}{a} \rfloor) \times (m \bmod a)^{-1} \pmod{m}$
- Fermat's little theorem: $a^p \equiv a \pmod{p}$ if p is prime.
- Euler function: $\varphi(n) = n \prod_{p|n} \frac{p-1}{p}$
- Euler theorem: $a^{\varphi(n)} \equiv 1 \pmod{n}$ if $\gcd(a, n) = 1$. If a, n are not coprime: 質因數分解 $n = \prod p_i^{e_i}$ ，對每個 $p_i^{e_i}$ 分開看他們跟 a 是否互質（互質：Fermat / 不互質：夠大的指數會直接削成 0），最後用 CRT 合併。
- Extended Euclidean algorithm: $ax + by = \gcd(a, b) = \gcd(b, a \bmod b) = \gcd(b, a - \lfloor \frac{a}{b} \rfloor b) = bx_1 + (a - \lfloor \frac{a}{b} \rfloor b)y_1 = ay_1 + b(x_1 - \lfloor \frac{a}{b} \rfloor y_1)$
- Divisor function: $\sigma_x(n) = \sum_{d|n} d^x$. $n = \prod_{i=1}^r p_i^{a_i}$.
 $\sigma_x(n) = \prod_{i=1}^r \frac{p_i^{(a_i+1)x} - 1}{p_i^x - 1}$ if $x \neq 0$. $\sigma_0(n) = \prod_{i=1}^r (a_i + 1)$.

- Chinese remainder theorem (Coprime Moduli):
 $x \equiv a_i \pmod{m_i}$.
 $M = \prod m_i$. $M_i = \frac{M}{m_i}$. $t_i = M_i^{-1}$.
 $x = kM + \sum a_i t_i M_i$, $k \in \mathbb{Z}$.
- Chinese remainder theorem:
 $x \equiv a_1 \pmod{m_1}, x \equiv a_2 \pmod{m_2} \Rightarrow x = m_1 p + a_1 = m_2 q + a_2 \Rightarrow m_1 p - m_2 q = a_2 - a_1$
Solve for (p, q) using ExtGCD.
 $x \equiv m_1 p + a_1 \equiv m_2 q + a_2 \pmod{\text{lcm}(m_1, m_2)}$
- Avoiding Overflow: $ca \bmod cb = c(a \bmod b)$
- Dirichlet Convolution: $(f * g)(n) = \sum_{d|n} f(d)g(\frac{n}{d})$
- Important Multiplicative Functions + Properties:
 - $\varepsilon(n) = [n = 1]$
 - $1(n) = 1$
 - $id(n) = n$
 - $\mu(n) = 0$ if n has squared prime factor
 - $\mu(n) = (-1)^k$ if $n = p_1 p_2 \dots p_k$
 - $\varepsilon = \mu * 1$
 - $\varphi = \mu * id$
 - $[n = 1] = \sum_{d|n} \mu(d)$
 - $[gcd = 1] = \sum_{d|gcd} \mu(d)$
- Möbius inversion: $f = g * 1 \Leftrightarrow g = f * \mu$

12.15 Polynomial

57318b

```

1 // Author: Gino
2 // Preparation: first set_mod(mod, g), then init_ntt()
3 // everytime you change modulus, you have to call
4 // init_ntt() again
5 // [Usage]
6 // polynomial: vector<ll> a, b
7 // negation: -a
8 // add/subtract: a += b, a -= b
9 // convolution: a *= b
10 // in-place modulo: mod(a, b)
11 // in-place inversion under mod x^N: inv(ia, N)
12
13
14 const int maxk = 20;
15 const int maxn = 1<<maxk;
16
17 using u64 = unsigned long long;
18 using u128 = __uint128_t;
19
20 int g;
21 u64 MOD;
22 u64 BARRETT_IM; // [ 2^64 / MOD ]
23
24 inline void set_mod(u64 m, int _g) {
25     g = _g;
26     MOD = m;
27     BARRETT_IM = (u128(1) << 64) / m;
28 }
29 inline u64 chmod(u128 x) {
30     u64 q = (u64)((x * BARRETT_IM) >> 64);
31     u64 r = (u64)(x - (u128)q * MOD);
32     if (r >= MOD) r -= MOD;
33     return r;
34 }
35 inline u64 mmul(u64 a, u64 b) {
36     return chmod((u128)a * b);
37 }
38 ll pw(ll a, ll n) {
39     ll ret = 1;
40     while (n > 0) {
41         if (n & 1) ret = mmul(ret, a);
42         a = mmul(a, a);
43         n >>= 1;
44     }
45     return ret;
46 }
47
48 vector<ll> X, iX;
49 vector<int> rev;
50 void init_ntt() {
51     X.assign(maxn, 1); // x1 = g^((p-1)/n)
52     iX.assign(maxn, 1);
53
54     ll u = pw(g, (MOD-1)/maxn);
55     ll iu = pw(u, MOD-2);
56     for (int i = 1; i < maxn; i++) {
57         X[i] = mmul(X[i-1], u);
58         iX[i] = mmul(iX[i-1], iu);
59     }
60
61     if ((int)rev.size() == maxn) return;

```

```

62 rev.assign(maxn, 0);
63 for (int i = 1, hb = -1; i < maxn; i++) {
64     if (!(i & (i-1))) hb++;
65     rev[i] = rev[i ^ (1<<hb)] | (1<<(maxk-hb-1));
66 }
67 template<typename T>
68 void NTT(vector<T>& a, bool inv=false) {
69     int n = (int)a.size();
70     int k = __lg(n) + ((1<<__lg(n)) != n);
71     int n = 1<<k;
72     a.resize(n, 0);
73
74     short shift = maxk-k;
75     for (int i = 0; i < n; i++)
76         if (i > (rev[i]>>shift))
77             swap(a[i], a[rev[i]>>shift]);
78     for (int len = 2, half = 1, div = maxn>>1; len <= n;
79         len<<=1, half<<=1, div>>=1) {
80         for (int i = 0; i < n; i += len) {
81             for (int j = 0; j < half; j++) {
82                 T u = a[i+j];
83                 T v = mmul(a[i+j+half], (inv ? iX[j*div] :
84                     X[j*div]));
85                 a[i+j] = (u+v >= MOD ? u+v-MOD : u+v);
86                 a[i+j+half] = (u-v < 0 ? u-v+MOD : u-v);
87             }
88         }
89         if (inv) {
90             T dn = pw(n, MOD-2);
91             for (auto& x : a) {
92                 x = mmul(x, dn);
93             }
94         }
95     }
96 }
97 template<typename T>
98 inline void shrink(vector<T>& a) {
99     int cnt = (int)a.size();
100     for (; cnt > 0; cnt--) if (a[cnt-1]) break;
101     a.resize(max(cnt, 1));
102 }
103 template<typename T>
104 vector<T>& operator+=(vector<T>& a, vector<T> b) {
105     int na = (int)a.size();
106     int nb = (int)b.size();
107     a.resize(na + nb - 1, 0);
108     b.resize(na + nb - 1, 0);
109     NTT(a); NTT(b);
110     for (int i = 0; i < (int)a.size(); i++)
111         a[i] = mmul(a[i], b[i]);
112     NTT(a, true);
113     shrink(a);
114     return a;
115 }
116 inline ll crt(ll a0, ll a1, ll m1, ll m2, ll
117     inv_m1_mod_m2){
118     // x ≡ a0 (mod m1), x ≡ a1 (mod m2)
119     // t = (a1 - a0) * inv(m1) mod m2
120     // x = a0 + t * m1 (mod m1*m2)
121     ll t = chmod(a1 - a0);
122     if (t < 0) t += m2;
123     t = (ll)((__int128)t * inv_m1_mod_m2 % m2);
124     return a0 + (ll)((__int128)t * m1);
125 }
126 void mul_crt() {
127     // a copy to a1, a2 | b copy to b1, b2
128     ll M1 = 998244353, M2 = 1004535809;
129     g = 3; set_mod(M1); init_ntt(); a1 *= b1;
130     g = 3; set_mod(M2); init_ntt(); a2 *= b2;
131
132     ll inv_m1_mod_m2 = pw(M1, M2 - 2);
133     for (int i = 2; i <= 2 * k; i++)
134         cout << crt(a1[i], a2[i], M1, M2, inv_m1_mod_m2)
135         << ' ';
136     cout << endl;
137 }
138 /* P = r*2^k + 1
139 P      r      k      g
140 998244353      119 23 3
141 1004535809      479 21 3
142
143 P      r      k      g
144 3      1      1 2
145 5      1      2 2
146 17     1      4 3
147 97     3      5 5
148 193    3      6 5
149 257    1      8 3
150 7681   15     9 17
151 12289  3     12 11

```

```

147 40961      5 13 3
148 65537     1 16 3
149 786433    3 18 10
150 5767169   11 19 3
151 7340033   7 20 3
152 23068673  11 21 3
153 104857601 25 22 3
154 167772161 5 25 3
155 469762049 7 26 3
156 1004535809 479 21 3
157 2013265921 15 27 31
158 2281701377 17 27 3
159 3221225473 3 30 5
160 75161927681 35 31 3
161 77309411329 9 33 7
162 206158430209 3 36 22
163 2061584302081 15 37 7
164 2748779069441 5 39 3
165 6597069766657 3 41 5
166 39582418599937 9 42 5
167 79164837199873 9 43 5
168 263882790666241 15 44 7
169 1231453023109121 35 45 3
170 1337006139375617 19 46 3
171 3799912185593857 27 47 5
172 4222124650659841 15 48 19
173 7881299347898369 7 50 6
174 31525197391593473 7 52 3
175 180143985094819841 5 55 6
176 1945555039024054273 27 56 5
177 4179340454199820289 29 57 3
178 9097271247288401921 505 54 6 */

```

12.16 Counting Primes

1662d0

```

1 // prime_count - #primes in [1..n] (O(n^{2/3}) time,
2 // O(sqrt(n)) memory)
3 using u64 = unsigned long long;
4 static inline u64 prime_count(u64 n){
5     if(n<=1) return 0;
6     int v = (int)floor(sqrt((long double)n));
7     int s = (v+1)>>1, pc = 0;
8     vector<int> smalls(s), roughs(s), skip(v+1);
9     vector<long long> larges(s);
10
11     for(int i=0;i<s;++i){
12         smalls[i]=i;
13         roughs[i]=2*i+1;
14         larges[i]=(long long)((n/roughs[i]-1)>>1);
15     }
16
17     for(int p=3;p<=v;p+=2) if(!skip[p]){
18         int q = p*p;
19         if(1LL*q*q > (long long)n) break;
20         skip[p]=1;
21         for(int i=q;i<=v;i+=2*p) skip[i]=1;
22
23         int ns=0;
24         for(int k=0;k<s;++k){
25             int i = roughs[k];
26             if(skip[i]) continue;
27             u64 d = (u64)i * (u64)p;
28             long long sub = (d <= (u64)v)
29                 ? larges[smalls[(int)(d>>1)] - pc]
30                 : smalls[(int)((n/d - 1) >> 1)];
31             larges[ns] = larges[k] - sub + pc;
32             roughs[ns++] = i;
33         }
34         s = ns;
35         for(int i=(v-1)>>1, j=((v/p)-1)|1; j>=p; j-=2){
36             int c = smalls[j>>1] - pc;
37             for(int e=(j*p)>>1; i>=e; --i) smalls[i] -= c;
38         }
39         ++pc;
40     }
41
42     larges[0] += 1LL*(s + 2*(pc-1))*(s-1) >> 1;
43     for(int k=1;k<s;++k) larges[0] -= larges[k];
44
45     for(int l=1;l<s;++l){
46         int q = roughs[l];
47         u64 m = n / (u64)q;
48         long long t = 0;
49         int e = smalls[(int)((m/q - 1) >> 1)] - pc;
50         if(e < l+1) break;
51         for(int k=l+1;k<=e;++k) t += smalls[(int)((m/
52             (u64)roughs[k] - 1) >> 1)];
53         larges[0] += t - 1LL*(e - l)*(pc + l - 1);
54     }
55     return (u64)(larges[0] + 1);

```

55 }

12.17 Linear Sieve for Other Number Theoretic Functions

d9fd0b

```
1// linear_sieve(n, primes, lp, phi, mu, d, sigma)
2// Outputs over the index range 0..n (n >= 1):
3// primes : all primes in [2..n], increasing.
4// lp      : lowest prime factor; lp[1]=0, lp[x] is the
5//           smallest prime dividing x.
6// phi     : Euler totient, phi[x] =  $\prod_{1 \leq k \leq x : \gcd(k,x)=1} 1$ . Multiplicative.
7// mu     : Möbius; mu[1]=1, mu[x]=0 if x has a squared
8//           prime factor, else  $(-1)^{\#\text{distinct primes}}$ .
9// d      : number of divisors; if  $x = \prod p_i^{e_i}$ , then
10//           $d[x] = \prod (e_i + 1)$ . Multiplicative.
11// sigma : sum of divisors; if  $x = \prod p_i^{e_i}$ , then
12//           $\sigma[x] = \prod (1 + p_i + \dots + p_i^{e_i})$ . (use ll)
13// Complexity:  $O(n)$  time,  $O(n)$  memory.
14// Notes: Arrays are resized inside; primes is cleared and
15//         reserved. sigma uses ll to avoid 32-bit overflow.
```

```
13 static inline void linear_sieve(
14     int n,
15     std::vector<int> &primes,
16     std::vector<int> &lp,
17     std::vector<int> &phi,
18     std::vector<int> &mu,
19     std::vector<int> &d,
20     std::vector<ll> &sigma
21 ) {
22     lp.assign(n + 1, 0); phi.assign(n + 1, 0); mu.assign(n +
23     1, 0); d.assign(n + 1, 0); sigma.assign(n + 1, 0);
24     primes.clear(); primes.reserve(n > 1 ? n / 10 : 0);
25     std::vector<int> cnt(n + 1, 0), core(n + 1, 1);
26     std::vector<ll> p_pow(n + 1, 1), sum_p(n + 1, 1);
27     phi[1] = mu[1] = d[1] = sigma[1] = 1;
28     for (int i = 2; i <= n; ++i) {
29         if (!lp[i]) {
30             lp[i] = i; primes.push_back(i);
31             phi[i] = i - 1; mu[i] = -1; d[i] = 2;
32             cnt[i] = 1; p_pow[i] = i; core[i] = 1;
33             sum_p[i] = 1 + (ll)i; sigma[i] = sum_p[i];
34         }
35         for (int p : primes) {
36             long long ip = 1LL * i * p;
37             if (ip > n) break;
38             lp[ip] = p;
39             if (p == lp[i]) {
40                 cnt[ip] = cnt[i] + 1; p_pow[ip] = p_pow[i] * p;
41                 core[ip] = core[i];
42                 sum_p[ip] = sum_p[i] + p_pow[ip];
43                 phi[ip] = phi[i] * p; mu[ip] = 0;
44                 d[ip] = d[core[ip]] * (cnt[ip] + 1);
45                 sigma[ip] = sigma[core[ip]] * sum_p[ip];
46                 break; // critical for linear complexity
47             } else {
48                 cnt[ip] = 1; p_pow[ip] = p; core[ip] = i;
49                 sum_p[ip] = 1 + (ll)p;
50                 phi[ip] = phi[i] * (p - 1); mu[ip] = -mu[i];
51                 d[ip] = d[i] * 2;
52                 sigma[ip] = sigma[i] * sum_p[ip];
53             }
54         }
55     }
56     // Optional helper: factorize x in  $O(\log x)$  using lp
57     // (requires x in [2..n])
58     static inline std::vector<std::pair<int, int>>
59     factorize(int x, const std::vector<int> &lp) {
60         while (x > 1) {
61             int p = lp[x], e = 0;
62             do { x /= p; ++e; } while (x % p == 0);
63             res.push_back({p, e});
64         }
65         return res;
66     }
```

13 Numerical

13.1 Polynomials and recurrences

```
1/* Author: David Rydh, Per Austrin
2* Description: */
3struct Poly {
4    vector<double> a;
5    double operator()(double x) const {
6        double val = 0;
7        for (int i = sz(a); i--;) (val *= x) += a[i];
```

```
8        return val;
9    }
10    void diff() {
11        rep(i, 1, sz(a)) a[i-1] = i*a[i];
12        a.pop_back();
13    }
14    void divroot(double x0) {
15        double b = a.back(), c; a.back() = 0;
16        for (int i = sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b,
17        b=c;
18        a.pop_back();
19    };
20    /* Author: Per Austrin (License: CC0)
21    * Description: Finds the real roots to a polynomial.
22    * Usage: polyRoots({{2,-3,1}}, -1e9, 1e9) // solve  $x^2-3x+2 = 0$ 
23    * Time:  $O(n^2 \log(1/\epsilon))$  */
24    #include "Polynomial.h"
25    vector<double> polyRoots(Poly p, double xmin, double xmax) {
26        if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
27        vector<double> ret;
28        Poly der = p;
29        der.diff();
30        auto dr = polyRoots(der, xmin, xmax);
31        dr.push_back(xmin-1);
32        dr.push_back(xmax+1);
33        sort(all(dr));
34        rep(i, 0, sz(dr)-1) {
35            double l = dr[i], h = dr[i+1];
36            bool sign = p(l) > 0;
37            if (sign ^ (p(h) > 0)) {
38                rep(it, 0, 60) { // while (h - l > 1e-8)
39                    double m = (l + h) / 2, f = p(m);
40                    if ((f <= 0) ^ sign) l = m;
41                    else h = m;
42                }
43                ret.push_back((l + h) / 2);
44            }
45        }
46        return ret;
47    }
48    /* Author: Simon Lindholm (License: CC0)
49    * Description: Given  $n$  points  $(x[i], y[i])$ , computes an
50    *  $n-1$ -degree polynomial  $p$  such
51    * 3 passes through them:  $p(x) = a[0]*x^0 + \dots +$ 
52    *  $a[n-1]*x^{n-1}$ .
53    * For numerical precision, pick  $x[k] = c*\cos(k/(n-1)*\pi)$ ,
54    *  $k=0 \dots n-1$ .
55    * Time:  $O(n^2)$  */
56    typedef vector<double> vd;
57    vd interpolate(vd x, vd y, int n) {
58        vd res(n), tmp(n);
59        rep(k, 0, n-1) rep(i, k+1, n)
60            y[i] = (y[i] - y[k]) / (x[i] - x[k]);
61        double last = 0; tmp[0] = 1;
62        rep(k, 0, n) rep(i, 0, n) {
63            res[i] += y[k] * tmp[i];
64            swap(last, tmp[i]);
65            tmp[i] -= last * x[k];
66        }
67        return res;
68    }
69    /* Author: koukanni (License: CC0)
70    * Description: Lagrange interpolation for polynomial
71    * evaluation at integer points.
72    * 3 Given polynomial values  $f[0], f[1], \dots, f[n-1]$ , computes
73    *  $f(c)$ .
74    * Time:  $O(n)$ 
75    * Status: tested */
76    Mint lagrange_iota(const vector<Mint> &f, const int c) {
77        const int n = int(size(f));
78        if (c < n) {
79            return f[c];
80        }
81        vector<Mint> fac(n), ifac(n);
82        fac[0] = 1;
83        for (int i = 1; i < n; i++) fac[i] = fac[i-1] * i;
84        ifac[n-1] = fac[n-1].inv();
85        for (int i = n-1; i >= 1; i--) ifac[i-1] = ifac[i] *
86        i;
87        vector<Mint> g(n);
88        for (int i = 0; i < n; i++) {
89            g[i] = f[i] * ifac[i] * ifac[n-1-i];
90            if ((n-1-i) & 1) g[i] *= -1;
91        }
92        vector<Mint> lp(n+1), rp(n+1);
93        lp[0] = rp[n] = 1;
```

```

23 for (int i = 0; i < n; i++) lp[i + 1] = lp[i] * Mint(c -
i);
24 for (int i = n - 1; i >= 0; i--) rp[i] = rp[i + 1] *
Mint(c - i);
25 Mint ans = 0;
26 for (int i = 0; i < n; i++) ans += g[i] * lp[i] * rp[i +
1];
27 return ans;
28 }

/* Author: Lucian Bicsi (License: CC0)
2 * Description: Recovers any $n$-order linear recurrence
relation from the first
3 * $2n$ terms of the recurrence.
4 * Useful for guessing linear recurrences after brute-forcing
the first terms.
5 * Should work on any field, but numerical stability for
floats is not guaranteed.
6 * Output will have size $\le n$.
7 * Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}
8 * Time: $O(N^2)$, Status: brute-force-tested mod 5 for $n \le 5$
and all $s$ */
9 #include "../number-theory/ModPow.h"
10 vector<ll> berlekampMassey(vector<ll> s) {
11 int n = sz(s), L = 0, m = 0;
12 vector<ll> C(n), B(n), T;
13 C[0] = B[0] = 1;
14 ll b = 1;
15 rep(i, 0, n) { ++m;
16 ll d = s[i] % mod;
17 rep(j, 1, L+1) d = (d + C[j] * s[i - j]) % mod;
18 if (!d) continue;
19 T = C; ll coef = d * modpow(b, mod-2) % mod;
20 rep(j, m, n) C[j] = (C[j] - coef * B[j - m]) % mod;
21 if (2 * L > i) continue;
22 L = i + 1 - L; B = T; b = d; m = 0;
23 }
24 // assert(2 * L + 30 <= n);
25 C.resize(L + 1); C.erase(C.begin());
26 for (ll& x : C) x = (mod - x) % mod;
27 return C;
28 }

/* Author: Lucian Bicsi (License: CC0)
2 * Description: Generates the $k$'th term of an $n$-order
linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$,
4 * given $S[0] \dots [ge n-1]$ and $tr[0] \dots [n-1]$.
5 * Faster than matrix multiplication.
6 * Useful together with Berlekamp-Massey.
7 * Usage: linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci
number
8 * Time: $O(n^2 \log k)$
9 * Status: brute-force-tested mod 5 for $n \le 5$ */
10 const ll mod = 5; /** exclude-line */
11 typedef vector<ll> Poly;
12 ll linearRec(Poly S, Poly tr, ll k) {
13 int n = sz(tr);
14 auto combine = [&](Poly a, Poly b) {
15 Poly res(n * 2 + 1);
16 rep(i, 0, n+1) rep(j, 0, n+1)
17 res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
18 for (int i = 2 * n; i > n; --i) rep(j, 0, n)
19 res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) %
mod;
20 res.resize(n + 1);
21 return res;
22 };
23 Poly pol(n + 1), e(pol);
24 pol[0] = e[1] = 1;
25 for (++k; k; k /= 2) {
26 if (k % 2) pol = combine(pol, e);
27 e = combine(e, e);
28 }
29 ll res = 0;
30 rep(i, 0, n) res = (res + pol[i + 1] * S[i]) % mod;
31 return res;
32 }

```

13.2 Optimization

```

/* Author: Ulf Lundstrom (License: CC0)
2 * Description: Finds the argument minimizing the function
$f$ in the interval $[a,b]$
3 * assuming $f$ is unimodal on the interval, i.e. has only
one local minimum and no local
4 * maximum. The maximum error in the result is $eps$. Works
equally well for maximization
5 * with a small change in the code. See TernarySearch.h in
the Various chapter for a
6 * discrete version.
7 * Usage: double func(double x) { return 4+x+.3*x*x; }
8 * double xmin = gss(-1000,1000,func);

```

```

9 * Time: $O(\log((b-a) / \epsilon))$, Status: tested */
10 /// It is important for $r$ to be precise, otherwise we don't
necessarily maintain the inequality $a < x_1 < x_2 < b$.
11 double gss(double a, double b, double (*f)(double)) {
12 double r = (sqrt(5)-1)/2, eps = 1e-7;
13 double x1 = b - r*(b-a), x2 = a + r*(b-a);
14 double f1 = f(x1), f2 = f(x2);
15 while (b-a > eps)
16 if (f1 < f2) { //change to > to find maximum
17 b = x2; x2 = x1; f2 = f1;
18 x1 = b - r*(b-a); f1 = f(x1);
19 } else {
20 a = x1; x1 = x2; f1 = f2;
21 x2 = a + r*(b-a); f2 = f(x2);
22 }
23 return a;
24 }

/* Author: Simon Lindholm (License: CC0)
2 * Description: Poor man's optimization for unimodal
functions.
3 * Status: used with great success */
4 typedef array<double, 2> P;
5 template<class F> pair<double, P> hillClimb(P start, F f) {
6 pair<double, P> cur(f(start), start);
7 for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
8 rep(j, 0, 100) rep(dx, -1, 2) rep(dy, -1, 2) {
9 P p = cur.second;
10 p[0] += dx*jmp;
11 p[1] += dy*jmp;
12 cur = min(cur, make_pair(f(p), p));
13 }
14 }
15 return cur;
16 }

/* Author: Simon Lindholm (License: CC0)
2 * Description: Simple integration of a function over an
interval using
3 * Simpson's rule. The error should be proportional to $h^4$,
although in
4 * practice you will want to verify that the result is stable
to desired
5 * precision when epsilon changes.
6 * Status: mostly untested */
7 template<class F>
8 double quad(double a, double b, F f, const int n = 1000) {
9 double h = (b - a) / 2 / n, v = f(a) + f(b);
10 rep(i, 1, n*2)
11 v += f(a + i*h) * (i&1 ? 4 : 2);
12 return v * h / 3;
13 }

/* Author: Simon Lindholm (License: CC0)
2 * Description: Fast integration using an adaptive Simpson's
rule.
3 * Usage:
4 * double sphereVolume = quad(-1, 1, [](double x) {
5 * return quad(-1, 1, [&](double y) {
6 * return quad(-1, 1, [&](double z) {
7 * return x*x + y*y + z*z < 1; });});});
8 Status: mostly untested */
9 typedef double d;
10 #define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6
11 template<class F>
12 d rec(F& f, d a, d b, d eps, d S) {
13 d c = (a + b) / 2;
14 d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
15 if (abs(T - S) <= 15 * eps || b - a < 1e-10)
16 return T + (T - S) / 15;
17 return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2,
S2);
18 }
19 template<class F>
20 d quad(d a, d b, F f, d eps = 1e-8) {
21 return rec(f, a, b, eps, S(a, b));
22 }

/* Author: Stanford (License: MIT)
2 * Description: Solves a general linear maximization problem:
maximize $c^T x$ subject to $Ax \le b$, $x \ge 0$.
3 * Returns -inf if there is no solution, inf if there are
arbitrarily good solutions, or the maximum value of $c^T x$
otherwise.
4 * The input vector is set to an optimal $x$ (or in the
unbounded case, an arbitrary solution fulfilling the
constraints).
5 * Numerical stability is not guaranteed. For better
performance, define variables such that $x = 0$ is viable.
6 * Usage:
7 * vvd A = {{1, -1}, {-1, 1}, {-1, -2}};

```

```

8 * vd b = {1,1,-4}, c = {-1,-1}, x;
9 * T val = LPSolver(A, b, c).solve(x);
10 * Time:  $O(NM \cdot \#\text{pivots})$ , where a pivot may be e.g. an edge
    relaxation.  $O(2^n)$  in the general case.
11 * Status: seems to work? */
12 typedef double T; // long double, Rational, double +
    mod<P>...
13 typedef vector<T> vd;
14 typedef vector<vd> vvd;
15 const T eps = 1e-8, inf = 1/.0;
16 #define MP make_pair
17 #define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s]))
    s=j
18 struct LPSolver {
19     int m, n; vi N, B; vvd D;
20     LPSolver(const vvd& A, const vd& b, const vd& c) :
21         m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
22         rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
23         rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] =
24             b[i]; }
25         rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
26         N[n] = -1; D[m+1][n] = 1;
27     }
28     void pivot(int r, int s) {
29         T *a = D[r].data(), inv = 1 / a[s];
30         rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
31             T *b = D[i].data(), inv2 = b[s] * inv;
32             rep(j,0,n+2) b[j] -= a[j] * inv2;
33             b[s] = a[s] * inv2;
34         }
35         rep(j,0,n+2) if (j != s) D[r][j] *= inv;
36         rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
37         D[r][s] = inv;
38         swap(B[r], N[s]);
39     }
40     bool simplex(int phase) {
41         int x = m + phase - 1;
42         for (;;) {
43             int s = -1;
44             rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
45             if (D[x][s] >= -eps) return true;
46             int r = -1;
47             rep(i,0,m) {
48                 if (D[i][s] <= eps) continue;
49                 if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
50                     < MP(D[r][n+1] / D[r][s], B[r])) r = i;
51             }
52             if (r == -1) return false;
53             pivot(r, s);
54         }
55     }
56     T solve(vd &x) {
57         int r = 0;
58         rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
59         if (D[r][n+1] < -eps) {
60             pivot(r, n);
61             if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
62             rep(i,0,m) if (B[i] == -1) {
63                 int s = 0;
64                 rep(j,1,n+1) ltj(D[i]);
65                 pivot(i, s);
66             }
67         }
68         bool ok = simplex(1); x = vd(n);
69         rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
70         return ok ? D[m][n+1] : inf;
71     }

```

13.3 Matrices

```

1 /* Author: Simon Lindholm (License: CC0)
2 * Description: Calculates determinant of a matrix. Destroys
    the matrix.
3 * Time:  $O(N^3)$ , Status: somewhat tested */
4 double det(vector<vector<double>>& a) {
5     int n = sz(a); double res = 1;
6     rep(i,0,n) {
7         int b = i;
8         rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
9         if (i != b) swap(a[i], a[b]), res *= -1;
10        res *= a[i][i];
11        if (res == 0) return 0;
12        rep(j,i+1,n) {
13            double v = a[j][i] / a[i][i];
14            if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
15        }
16    }
17    return res;
18 }

```

```

1 /* Author: Unknown
2 * Description: Calculates determinant using modular
    arithmetics.
3 * Modulos can also be removed to get a pure-integer version.
4 * Time:  $O(N^3)$ 
5 * Status: brute-force-tested for  $N \leq 3$ , mod  $\leq 7$  */
6 const ll mod = 12345;
7 ll det(vector<vector<ll>>& a) {
8     int n = sz(a); ll ans = 1;
9     rep(i,0,n) {
10         rep(j,i+1,n) {
11             while (a[j][i] != 0) { // gcd step
12                 ll t = a[i][i] / a[j][i];
13                 if (t) rep(k,i,n)
14                     a[i][k] = (a[i][k] - a[j][k] * t) % mod;
15                 swap(a[i], a[j]);
16                 ans *= -1;
17             }
18         }
19         ans = ans * a[i][i] % mod;
20         if (!ans) return 0;
21     }
22     return (ans + mod) % mod;
23 }

```

```

1 /* Author: Per Austrin, Simon Lindholm (License: CC0)
2 * Description: Solves  $A \cdot x = b$ . If there are multiple
    solutions, an arbitrary one is returned.
3 * Returns rank, or -1 if no solutions. Data in  $A$  and  $b$ 
    is lost.
4 * Time:  $O(n^2 m)$ 
5 * Status: tested on kattis:equationsolver, and brute-force-
    tested mod 3 and 5 for  $n, m \leq 3$  */
6 typedef vector<double> vd;
7 const double eps = 1e-12;
8 int solveLinear(vector<vd>& A, vd& b, vd& x) {
9     int n = sz(A), m = sz(x), rank = 0, br, bc;
10    if (n) assert(sz(A[0]) == m);
11    vi col(m); iota(all(col), 0);
12    rep(i,0,n) {
13        double v, bv = 0;
14        rep(r,i,n) rep(c,i,m)
15            if ((v = fabs(A[r][c])) > bv)
16                br = r, bc = c, bv = v;
17        if (bv <= eps) {
18            rep(j,i,n) if (fabs(b[j]) > eps) return -1;
19            break;
20        }
21        swap(A[i], A[br]);
22        swap(b[i], b[br]);
23        swap(col[i], col[bc]);
24        rep(j,0,n) swap(A[j][i], A[j][bc]);
25        bv = 1/A[i][i];
26        rep(j,i+1,n) {
27            double fac = A[j][i] * bv;
28            b[j] -= fac * b[i];
29            rep(k,i+1,m) A[j][k] -= fac * A[i][k];
30        }
31        rank++;
32    }
33    x.assign(m, 0);
34    for (int i = rank; i--;) {
35        b[i] /= A[i][i];
36        x[col[i]] = b[i];
37        rep(j,0,i) b[j] -= A[j][i] * b[i];
38    }
39    return rank; // (multiple solutions if rank < m)
40 }

```

```

1 /* Author: Simon Lindholm (License: CC0)
2 * Description: To get all uniquely determined values of  $x$ 
    back from SolveLinear, make the following changes:
3 * Status: tested on kattis:equationsolverplus, stress-tested
    */
4 #include "SolveLinear.h"
5 rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
6 // ... then at the end:
7 x.assign(m, undefined);
8 rep(i,0,rank) {
9     rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
10    x[col[i]] = b[i] / A[i][i];
11 fail; }
12
1 /* Author: Simon Lindholm (License: CC0)
2 * Description: Solves  $Ax = b$  over  $\mathbb{F}_2$ . If there
    are multiple solutions, one is returned arbitrarily.
3 * Returns rank, or -1 if no solutions. Destroys  $A$  and  $b$ .
4 * Time:  $O(n^2 m)$ 
5 * Status: brute-force-tested for  $n, m \leq 4$  */
6 typedef bitset<1000> bs;
7 int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {

```

```

8  int n = sz(A), rank = 0, br;
9  assert(m <= sz(x));
10 vi col(m); iota(all(col), 0);
11 rep(i,0,n) {
12   for (br=i; br<n; ++br) if (A[br].any()) break;
13   if (br == n) {
14     rep(j,i,n) if(b[j]) return -1;
15     break;
16   }
17   int bc = (int)A[br]._Find_next(i-1);
18   swap(A[i], A[br]);
19   swap(b[i], b[br]);
20   swap(col[i], col[bc]);
21   rep(j,0,n) if (A[j][i] != A[j][bc]) {
22     A[j].flip(i); A[j].flip(bc);
23   }
24   rep(j,i+1,n) if (A[j][i]) {
25     b[j] ^= b[i];
26     A[j] ^= A[i];
27   }
28   rank++;
29 }
30 x = bs();
31 for (int i = rank; i--;) {
32   if (!b[i]) continue;
33   x[col[i]] = 1;
34   rep(j,0,i) b[j] ^= A[j][i];
35 }
36 return rank; // (multiple solutions if rank < m)
37 }

```

1/* Author: Max Bennedich

2* Description: Invert matrix \$A\$. Returns rank; result is stored in \$A\$ unless singular (rank < n).
3* Can easily be extended to prime moduli; for prime powers, repeatedly
4* set \$A^{-1} = A^{-1} (2I - AA^{-1}) \pmod{p^k}\$ where \$A^{-1}\$ starts as
5* the inverse of \$A \pmod{p}\$, and \$k\$ is doubled in each step.
6* Time: \$O(n^3)\$
7* Status: Slightly tested */

```

8 int matInv(vector<vector<double>>& A) {
9   int n = sz(A); vi col(n);
10  vector<vector<double>> tmp(n, vector<double>(n));
11  rep(i,0,n) tmp[i][i] = 1, col[i] = i;
12  rep(i,0,n) {
13   int r = i, c = i;
14   rep(j,i,n) rep(k,i,n)
15     if (fabs(A[j][k]) > fabs(A[r][c]))
16       r = j, c = k;
17   if (fabs(A[r][c]) < 1e-12) return i;
18   A[i].swap(A[r]); tmp[i].swap(tmp[r]);
19   rep(j,0,n)
20     swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
21   swap(col[i], col[c]);
22   double v = A[i][i];
23   rep(j,i+1,n) {
24     double f = A[j][i] / v;
25     A[j][i] = 0;
26     rep(k,i+1,n) A[j][k] -= f*A[i][k];
27     rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
28   }
29   rep(j,i+1,n) A[i][j] /= v;
30   rep(j,0,n) tmp[i][j] /= v;
31   A[i][i] = 1;
32 }
33 // forget A at this point, just eliminate tmp backward
34 for (int i = n-1; i > 0; --i) rep(j,0,i) {
35   double v = A[j][i];
36   rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
37 }
38 rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
39 return n;
40 }

```

1/* Author: Simon Lindholm

2* Description: Invert matrix \$A\$ modulo a prime.
3* Returns rank; result is stored in \$A\$ unless singular (rank < n).
4* For prime powers, repeatedly set \$A^{-1} = A^{-1} (2I - AA^{-1}) \pmod{p^k}\$ where \$A^{-1}\$ starts as
5* the inverse of \$A \pmod{p}\$, and \$k\$ is doubled in each step.
6* Time: \$O(n^3)\$
7* Status: Slightly tested */

```

8 #include "../number-theory/ModPow.h"
9 int matInv(vector<vector<ll>>& A) {
10  int n = sz(A); vi col(n);
11  vector<vector<ll>> tmp(n, vector<ll>(n));
12  rep(i,0,n) tmp[i][i] = 1, col[i] = i;

```

```

13 rep(i,0,n) {
14   int r = i, c = i;
15   rep(j,i,n) rep(k,i,n) if (A[j][k]) {
16     r = j; c = k; goto found;
17   }
18   return i;
19 found:
20   A[i].swap(A[r]); tmp[i].swap(tmp[r]);
21   rep(j,0,n)
22     swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
23   swap(col[i], col[c]);
24   ll v = modpow(A[i][i], mod - 2);
25   rep(j,i+1,n) {
26     ll f = A[j][i] * v % mod;
27     A[j][i] = 0;
28     rep(k,i+1,n) A[j][k] = (A[j][k] - f*A[i][k]) % mod;
29     rep(k,0,n) tmp[j][k] = (tmp[j][k] - f*tmp[i][k]) %
30     mod;
31   }
32   rep(j,i+1,n) A[i][j] = A[i][j] * v % mod;
33   rep(j,0,n) tmp[i][j] = tmp[i][j] * v % mod;
34   A[i][i] = 1;
35 }
36 for (int i = n-1; i > 0; --i) rep(j,0,i) {
37   ll v = A[j][i];
38   rep(k,0,n) tmp[j][k] = (tmp[j][k] - v*tmp[i][k]) % mod;
39 }
40 rep(i,0,n) rep(j,0,n)
41   A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i][j] <
42   0)*mod;
43 return n;
44 }

```

1/* Author: Ulf Lundstrom, Simon Lindholm (License: CC0)

2* Description: Solves the tridiagonal matrix equation system:

```

3 * [ b_0 ] [ d_0 p_0 0 0 ... 0 ] [ x_0 ]
4 * [ b_1 ] [ q_0 d_1 p_1 0 ... 0 ] [ x_1 ]
5 * [ b_2 ] [ 0 q_1 d_2 p_2 ... 0 ] [ x_2 ]
6 * [ ... ] [ ... ... ... ... ... ] [ ... ]
7 * [ b_{n-1} ] [ 0 0 ... 0 q_{n-2} d_{n-1} ] [ x_{n-1} ]

```

8 * Application:

9 * Solves linear recurrence systems of the form:
10 * $a_i = b_i * a_{i-1} + c_i * a_{i+1} + d_i$ ($1 \leq i \leq n$)
11 * where a_0, a_{n+1}, b_i, c_i , and d_i are given.
12 * The solution vector a can be obtained via:
13 * $d = \{ 1, -1, -1, \dots, -1, 1 \}$
14 * $p = \{ 0, c_1, c_2, \dots, c_n \}$
15 * $q = \{ b_1, b_2, \dots, b_n, 0 \}$
16 * $b = \{ a_0, d_1, d_2, \dots, d_n, a_{n+1} \}$

17 * Numerical Stability:

18 * - Fails if the solution is not unique.
19 * - Numerically stable without pivot checks if:
20 * 1) Strictly diagonally dominant:
21 * $|d_i| > |p_i| + |q_{i-1}|$ for all i , OR
22 * $|d_i| > |p_{i-1}| + |q_i|$ for all i
23 * 2) The matrix is symmetric positive-definite.
24 * Complexity: $O(N)$ time, $O(N)$ space. Status: Brute-force tested mod 5 and 7, stress-tested for real matrices. */

```

25 typedef double T;
26 vector<T> tridiagonal(vector<T> diag, const vector<T>&
27   super,
28   const vector<T>& sub, vector<T> b) {
29   int n = sz(b); vi tr(n);
30   rep(i,0,n-1) {
31     if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] ==
32       0
33       b[i+1] -= b[i] * diag[i+1] / super[i];
34       if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
35       diag[i+1] = sub[i]; tr[i+1] = 1;
36     } else {
37       diag[i+1] -= super[i]*sub[i]/diag[i];
38       b[i+1] -= b[i]*sub[i]/diag[i];
39     }
40   }
41   for (int i = n; i--;) {
42     if (tr[i]) {
43       swap(b[i], b[i-1]);
44       diag[i-1] = diag[i];
45       b[i] /= super[i-1];
46     } else {
47       b[i] /= diag[i];
48       if (i) b[i-1] -= b[i]*super[i-1];
49     }
50   }
51   return b;
52 }

```

13.4 Fourier transforms

```

1/**
2 * Author: chilli, Ludo Pulles, Simon Lindholm (License:
  CC0)
3 * Description: Higher precision FFT, can be used for
  convolutions modulo arbitrary integers
4 * as long as N\log_2N * mod < 8.6e14 (in practice 10^16 or
  higher).
5 * Inputs must be in [0, mod).
6 * Time: O(N \log N), where N = |A| + |B| (twice as slow as
  NTT or normal FFT), Status: stress-tested */
7
8typedef complex<double> C;
9typedef vector<ll> vl;
10void fft(vector<C>& a) {
11    int n = sz(a), L = 31 - __builtin_clz(n);
12    static vector<complex<long double>> R(2, 1);
13    static vector<C> rt(2, 1);
14    for (static int k = 2; k < n; k *= 2) {
15        R.resize(n); rt.resize(n);
16        auto x = polar(1.0L, acos(-1.0L) / k);
17        rep(i, k, 2 * k) rt[i] = R[i] = i & 1 ? R[i / 2] * x :
  R[i / 2];
18    }
19    vi rev(n);
20    rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
21    rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
22    for (int k = 1; k < n; k *= 2)
23        for (int i = 0; i < n; i += 2 * k) rep(j, 0, k) {
24            auto x = (double *)&rt[j + k], y = (double *)&a[i + j
  + k];
25            C z(x[0] * y[0] - x[1] * y[1], x[0] * y[1] + x[1] *
  y[0]);
26            a[i + j + k] = a[i + j] - z;
27            a[i + j] += z;
28        }
29    }
30    template<int M> vl convMod(const vl &a, const vl &b) {
31        if (a.empty() || b.empty()) return {};
32        vl res(sz(a) + sz(b) - 1);
33        int B = 32 - __builtin_clz(sz(res)), n = 1 << B, cut =
  int(sqrt(M));
34        vector<C> L(n), R(n), outs(n), outl(n);
35        rep(i, 0, sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] %
  cut);
36        rep(i, 0, sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] %
  cut);
37        fft(L), fft(R);
38        rep(i, 0, n) {
39            int j = -i & (n - 1);
40            outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
41            outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
42        }
43        fft(outl), fft(outs);
44        rep(i, 0, sz(res)) {
45            ll av = ll(real(outl[i]) + .5), cv = ll(imag(outs[i])
  + .5);
46            ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i]) + .5);
47            res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
48        }
49        return res;
50    }
51
52    /* Author: Lucian Bicsi (License: GNU Free Documentation
  License 1.2)
53    * Description: Transform to a basis with fast convolutions
  of the form
54    * c[z] = \sum_{z = x \oplus y} a[x] * b[y]
55    * where \oplus is one of AND, OR, XOR.
56    * The size of $a$ must be a power of 2.
57    * Time: O(N \log N), Status: stress-tested */
58    void FST(vi& a, bool inv) {
59        for (int n = sz(a), step = 1; step < n; step *= 2) {
60            for (int i = 0; i < n; i += 2 * step) rep(j, i, i+step) {
61                int &u = a[j], &v = a[j + step]; tie(u, v) =
62                inv ? pii(v - u, u) : pii(v, u + v); // AND
63                // inv ? pii(v, u - v) : pii(u + v, u); // OR ///
64                include-line
65                // pii(u + v, u - v); // XOR ///
66                include-line
67            }
68        }
69        // if (inv) for (int& x : a) x /= sz(a); // XOR only ///
70        include-line
71    }
72    vi conv(vi a, vi b) {
73        FST(a, 0); FST(b, 0);
74        rep(i, 0, sz(a)) a[i] *= b[i];
75        FST(a, 1); return a;
76    }
77}

```

14 Combinatorics

14.1 Catalan Number

$$C_0 = 1, C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}, C_n = C_n^{2n} - C_{n-1}^{2n}$$

0	1	1	2	5
4	14	42	132	429
8	1430	4862	16796	58786
12	208012	742900	2674440	9694845

14.2 Bertrand's Ballot Theorem

- A always $> B$: $C(p+q, p) - 2C(p+q-1, p)$
- A always $\geq B$: $C(p+q, p) \times \frac{p+1-q}{p+1}$

14.3 Burnside's Lemma

Let X be the original set.

Let G be the group of operations acting on X .

Let X^g be the set of x not affected by g .

Let X/G be the set of orbits. Then the following equation holds:

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

15 Special Numbers

15.1 Fibonacci Series

1	1	1	2	3
5	5	8	13	21
9	34	55	89	144
13	233	377	610	987
17	1597	2584	4181	6765
21	10946	17711	28657	46368
25	75025	121393	196418	317811
29	514229	832040	1346269	2178309
33	3524578	5702887	9227465	14930352

$$f(45) \approx 10^9, f(88) \approx 10^{18}$$

15.2 Prime Numbers

- $\pi(n) \equiv$ Number of primes $\leq n \approx \frac{n}{(\ln n)-1}$
- $\pi(100) = 25, \pi(200) = 46$
- $\pi(500) = 95, \pi(1000) = 168$
- $\pi(2000) = 303, \pi(4000) = 550$
- $\pi(10^4) = 1229, \pi(10^5) = 9592$
- $\pi(10^6) = 78498, \pi(10^7) = 664579$

15.3 Number of Divisors

- If $n = \prod p_i^{a_i}$, then $\tau(n) = \prod (a_i + 1)$
- Maximum $\tau(n)$ for $n \leq 10^k$:

k	$\max \tau(n)$	k	$\max \tau(n)$	k	$\max \tau(n)$
3	32	9	1344	15	26880
4	64	10	2304	16	41472
5	128	11	4032	17	64512
6	240	12	6720	18	103680
7	448	13	10752		
8	768	14	17280		

- Useful bounds: $n \leq 10^6 \Rightarrow \tau(n) \leq 240$
- $n \leq 10^9 \Rightarrow \tau(n) \leq 1344$
- $n \leq 10^{12} \Rightarrow \tau(n) \leq 6720$
- $n \leq 10^{15} \Rightarrow \tau(n) \leq 26880$
- $n \leq 10^{18} \Rightarrow \tau(n) \leq 103680$

15.4 Distinct Prime Factors

- Let $\omega(n)$ = number of distinct prime factors.
- Minimum number with k distinct prime factors is the product of the first k primes.
 - $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 = 9699690$
 - $2 \cdot 3 \cdot \dots \cdot 23 = 223092870$
 - $2 \cdot 3 \cdot \dots \cdot 29 = 6469693230$
 - $n \leq 10^9 \Rightarrow \omega(n) \leq 9, n \leq 10^{18} \Rightarrow \omega(n) \leq 15$
- Number of square-free divisors = $2^{\omega(n)}$
 - $n \leq 10^9 \Rightarrow \leq 512, n \leq 10^{18} \Rightarrow \leq 32768$

